SQL Les bases

Laurent Bellon

2026

Bases de données relationnelles

Concepts fondamentaux

Qu’est-ce qu’une base de données relationnelle ?

- Système de gestion d’information structuré en **tables** :
  - *Lignes* = enregistrements
  - *Colonnes* = attributs
- Permet de représenter les données clairement et de façon cohérente
- Les tables sont liées par des **relations** via des clés primaires et étrangères
- Utilisation du langage standardisé **SQL** pour manipuler et interroger les données

Définition selon IBM et sources spécialisées

- Stockage des informations sous forme de tableaux bidimensionnels
- Chaque ligne = un enregistrement unique
- Chaque colonne = une caractéristique de l’enregistrement
- Relations entre tables assurent intégrité et cohérence
- Langage SQL pour accès et gestion des données

Tables principales :

- Livre (ID_Livre, Titre, Auteur, Année_Publication)
- Emprunteur (ID_Emprunteur, Nom, Prénom, Date_Inscription)
- Emprunt (ID_Emprunt, ID_Livre FK, ID_Emprunteur FK, Date_Emprunt, Date_Retour)

La table *Emprunt* relie *Livre* et *Emprunteur*

Avantages des bases relationnelles

- **Structure claire** avec tables normalisées
- **Intégrité des données** garantie par contraintes (clés primaires/étrangères)
- **Interrogation puissante** via SQL
- **Flexibilité** pour gérer des relations complexes
- **Maintenance et évolutivité** facilitées grâce aux standards

Sources

- Wikipédia, Base de données relationnelle
- IBM, Qu’est-ce qu’une base de données relationnelle
- Intersystems, Qu’est-ce qu’une base de données relationnelle et pourquoi en avez-vous besoin ?
- Google Cloud, Qu’est-ce qu’une base de données relationnelle (SGBDR)

Cette présentation introduit les notions clés des bases de données relationnelles : organisation, fonctionnement et bénéfices essentiels pour une gestion fiable et structurée des données.

Introduction aux bases de données relationnelles

Concepts fondamentaux des bases relationnelles

Modèle relationnel : tables, lignes, colonnes

Le modèle relationnel

- Modèle mathématique structurant les données en **relations** (tables).
- Chaque table regroupe des entités partageant des caractéristiques communes.

Structures fondamentales

- **Table (Relation)** : Ensemble organisé en lignes et colonnes.
- **Lignes (Tuples)** : Enregistrements uniques, regroupant les valeurs des attributs.
- **Colonnes (Attributs)** : Champs décrivant les propriétés des tuples, avec nom unique et type défini.

Exemple concret : Table Étudiant

ID_Etudiant	Nom	Prénom	Date_Naissance	Filière
1001	Dupont	Alice	2000-05-12	Informatique
1002	Martin	Bob	1999-11-08	Mathématiques
1003	Durand	Claire	2001-03-22	Physique

- **Table** : Étudiant
- **Colonnes** : ID_Etudiant, Nom, Prénom, Date_Naissance, Filière
- **Lignes** : Chaque étudiant (Alice, Bob, Claire) = un tuple

Visualisation du modèle (Mermaid)

ETUDIAN			
int	ID_Etudia	PK	Identifiant uni
strin	Non		Nom de l'étudi
strin	Prénon		Prénom de l'étud
date	Date_Naissan		Date de naissa
strin	Filièr		Filière de l'étud

Relations et contraintes

- **Clé primaire** : Identifie de façon unique chaque ligne (ex : `ID_Etudiant`).
- **Valeurs atomiques** : Chaque cellule contient une valeur indivisible.
- **Uniformité du type** : Une colonne contient un seul type de données.

Ces règles garantissent cohérence et facilitent les requêtes.

Synthèse

- Le modèle relationnel organise les données en tables composées de lignes (enregistrements) et colonnes (attributs).
- Cette structure simple permet d’assurer l’intégrité des données et de gérer les relations entre elles efficacement.

Introduction aux bases de données relationnelles

Concepts fondamentaux des bases relationnelles

Clés primaires et étrangères

Clés primaires et **clés étrangères** sont des éléments clés du modèle relationnel.

Elles assurent l’unicité des enregistrements et définissent les relations entre les tables.

Clé primaire (Primary Key)

- Un ou plusieurs attributs identifiant de façon unique chaque ligne d'une table.
- **Unique** : pas de duplication.
- **Non nulle** : ne peut pas contenir de valeur nulle.
- Assure l'**intégrité d'entité** (identification univoque).
- Peut être un identifiant naturel (ex. numéro de sécurité sociale) ou artificiel (identifiant généré).

Exemple dans une table *Employé* :

ID_Employe	Nom	Département
101	Dupont	Ventes
102	Martin	IT
103	Durand	RH

ID_Employe est la clé primaire.

Clé étrangère (Foreign Key)

- Un ou plusieurs attributs dans une table qui réfèrent à une clé primaire dans une autre table (ou la même table).
- Permet de **lier les données** entre tables.
- Maintient l’intégrité référentielle.

Contraintes

- La valeur doit exister dans la clé primaire de la table référencée, ou être nulle.
- Empêche les références vers des enregistrements inexistants (évite les données orphelines).

Exemple avec les tables *Employé* et *Projet* :

Employé

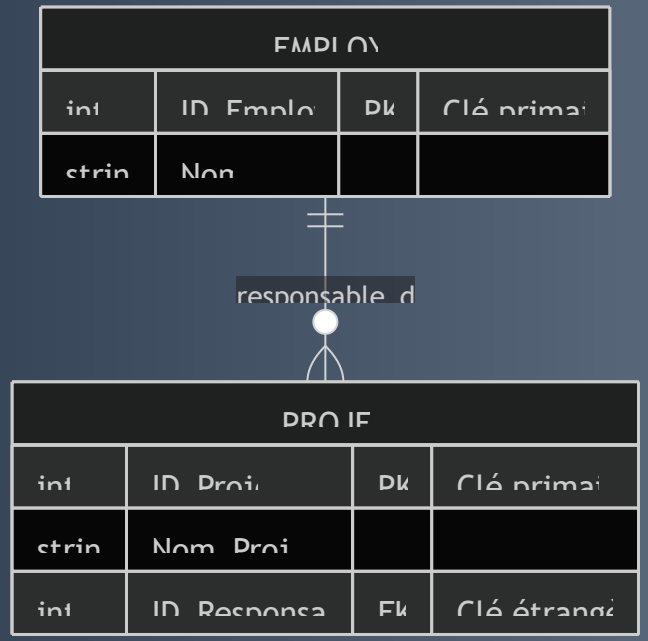
ID_Employe	Nom
101	Dupont
102	Martin

Projet

ID_Projet	Nom_Projet	ID_Responsable
501	Alpha	101
502	Beta	102

ID_Responsable est une clé étrangère faisant référence à *ID_Employe*.

Visualisation avec Mermaid



Synthèse

- **Clé primaire** : identifie de manière unique chaque ligne d'une table.
- **Clé étrangère** : crée des relations entre tables et assure l'intégrité des données.
- Ensemble, elles structurent la base relationnelle et facilitent les jointures et contraintes.

Sources

- Wikipédia : Clé primaire, Clé étrangère
- Oracle : Primary and Foreign Keys
- Microsoft Docs : Primary and Foreign Keys
- IBM : Database keys

Qu’est-ce que PostgreSQL ?

- Système de gestion de base de données relationnelle (SGBDR) open source
- Robuste, conforme aux standards
- Riche en fonctionnalités et performant
- Objectif : installer et configurer rapidement pour démarrer

Installation de PostgreSQL

Choix de la version

- Version recommandée en 2024 : PostgreSQL 15.x
- Améliorations :
  - Performances
  - Sécurité
  - Fonctionnalités

Installation sous Linux (Ubuntu)

```
sudo apt update
sudo apt install postgresql postgresql-contrib
```

- Installe PostgreSQL et modules additionnels

Installation sous Windows

- Télécharger l’installateur officiel PostgreSQL Downloads
- Suivre l’assistant d’installation
- Définir mot de passe de l’utilisateur `postgres`
- Choisir le port (par défaut 5432)

Installation sous macOS

```
brew update
brew install postgresql
brew services start postgresql
```

- Utilise Homebrew
- Lance le service PostgreSQL automatiquement

Configuration de base

Démarrer le service PostgreSQL sous Linux

```
sudo systemctl start postgresql
sudo systemctl enable postgresql  # démarrage automatique au boot
```

Se connecter au serveur PostgreSQL

```
sudo -i -u postgres
psql
```

- Connexion en super utilisateur `postgres`
- Accès à l'interface interactive SQL `psql`

Exemple d’utilisation initiale

Création d’une base de données

```
CREATE DATABASE formation;
```

Création d’un utilisateur

```
CREATE USER alice WITH PASSWORD 'securePwd123';
```

Attribution de privilèges

```
GRANT ALL PRIVILEGES ON DATABASE formation TO alice;
```

Fichiers principaux de configuration

- **pg_hba.conf** : contrôles d’authentification et accès réseau
- **postgresql.conf** : configuration générale (ports, performance, logs)

Exemple d’autorisation dans pg_hba.conf

```
# TYPE  DATABASE        USER            ADDRESS                 METHOD
local   all             all                                     md5
```

- Permet connexion locale par mot de passe

Appliquer les modifications

```
sudo systemctl restart postgresql
```

- Redémarre le service pour prendre en compte les changements

Architecture simplifiée de PostgreSQL

Sources utilisées

- PostgreSQL Documentation 15
- Téléchargements officiels PostgreSQL
- DigitalOcean, How To Install and Use PostgreSQL on Ubuntu 20.04
- PostgreSQL Wiki - Configuring PostgreSQL

Conclusion

- Installation et configuration rapide
- Base solide pour gérer des bases de données relationnelles
- PostgreSQL : un SGBDR puissant et flexible pour vos projets

PostgreSQL : un SGBDR puissant

- Système de Gestion de Bases de Données Relationnelles
- Langage SQL standardisé et puissant
- Permet création, manipulation et interrogation des données efficientes

Commandes SQL de base dans PostgreSQL

Création de tables (`CREATE TABLE`)

```sql
CREATE TABLE employe (
    id SERIAL PRIMARY KEY,
    nom VARCHAR(50) NOT NULL,
    prenom VARCHAR(50),
    age INTEGER,
    salaire NUMERIC(10,2)
);
```

- `SERIAL` : entier auto-incrémenté, clé primaire
- `VARCHAR(n)` : texte jusqu'à n caractères
- `NUMERIC(10,2)` : nombre à 10 chiffres dont 2 décimales

Insertion de données (`INSERT INTO`)

```sql
INSERT INTO employe (nom, prenom, age, salaire) VALUES
('Dupont', 'Alice', 30, 3500.00),
('Martin', 'Bob', 45, 4200.50);
```

Lecture de données (`SELECT`)

Lire toutes les données :

```
SELECT * FROM employe;
```

Filtrer par condition :

```
SELECT nom, prenom FROM employe WHERE age > 40;
```

Mise à jour (`UPDATE`)

Modifier des données existantes :

```
UPDATE employe SET salaire = salaire * 1.05 WHERE age > 40;
```

Suppression (`DELETE`)

Supprimer des enregistrements :

```
DELETE FROM employe WHERE id = 2;
```

Types de données courants dans PostgreSQL

- `INTEGER`, `SMALLINT` : entiers
- `SERIAL` : entier auto-incrémenté
- `VARCHAR(n)`, `TEXT` : chaînes de caractères
- `NUMERIC(p,s)`, `FLOAT` : nombres décimaux
- `DATE`, `TIMESTAMP` : dates et horodatages
- `BOOLEAN` : vrai/faux

Exemple complet : Table **Produit**

```sql
CREATE TABLE produit (
    id SERIAL PRIMARY KEY,
    nom VARCHAR(100) NOT NULL,
    prix NUMERIC(8,2),
    en_stock BOOLEAN DEFAULT TRUE
);

INSERT INTO produit (nom, prix) VALUES
('Stylo', 1.20),
('Cahier', 2.50),
('Gomme', 0.80);

SELECT * FROM produit WHERE en_stock = true;
```

Diagramme Mermaid de la table Produit

PRODUI			
int	id	Pk	Identifiant unique auto-incré
strin	non		Nom du proc
numer	pri		Prix avec décim
boolea	en_stoc		Produit en stock ou

Remarques importantes

- Identifiants sensibles à la casse entre guillemets doubles `" "`
- Par défaut, noms convertis en minuscules
- Commandes SQL terminées par `;`
- Syntaxe SQL insensible à la casse, mots-clés en majuscules recommandés

Sources utilisées

- Documentation officielle PostgreSQL, SQL Syntax
- Tutorialspoint, PostgreSQL Tutorial
- DigitalOcean, Introduction to PostgreSQL
- W3Schools, PostgreSQL Tutorial

Bases solides en SQL sur PostgreSQL : création, lecture, mise à jour et suppression de données relationnelles. Maîtrise des types courants et syntaxe essentielle.

Requêtes SQL fondamentales

Les commandes de base SQL

Syntaxe SELECT pour interroger une table

La commande **SELECT** est la pierre angulaire du langage SQL, utilisée pour interroger une ou plusieurs tables et extraire les données. Comprendre sa syntaxe et ses options est fondamental pour manipuler les bases relationnelles.

Structure de base de la commande SELECT

```
SELECT [colonnes]
FROM [table]
[WHERE conditions]
[ORDER BY colonne [ASC|DESC]]
[LIMIT n];
```

- **SELECT** : colonnes à récupérer
- **FROM** : table source
- **WHERE** : filtre conditionnel
- **ORDER BY** : tri des résultats
- **LIMIT** : nombre maximum de lignes retournées

Exemple simple avec la table Clients

id	nom	ville	age
1	Dupont	Paris	35
2	Martin	Lyon	27
3	Leroy	Marseille	42

Requêtes représentatives :

- Tous les clients, toutes colonnes :

```
SELECT * FROM Clients;
```

- Noms et villes uniquement :

```
SELECT nom, ville FROM Clients;
```

- Clients de plus de 30 ans :

```
SELECT * FROM Clients WHERE age > 30;
```

- Tri par âge décroissant :

```
SELECT * FROM Clients ORDER BY age DESC;
```

- Deux clients les plus âgés :

```
SELECT * FROM Clients ORDER BY age DESC LIMIT 2;
```

Opérations spécifiques

- Éviter les doublons (villes distinctes) :

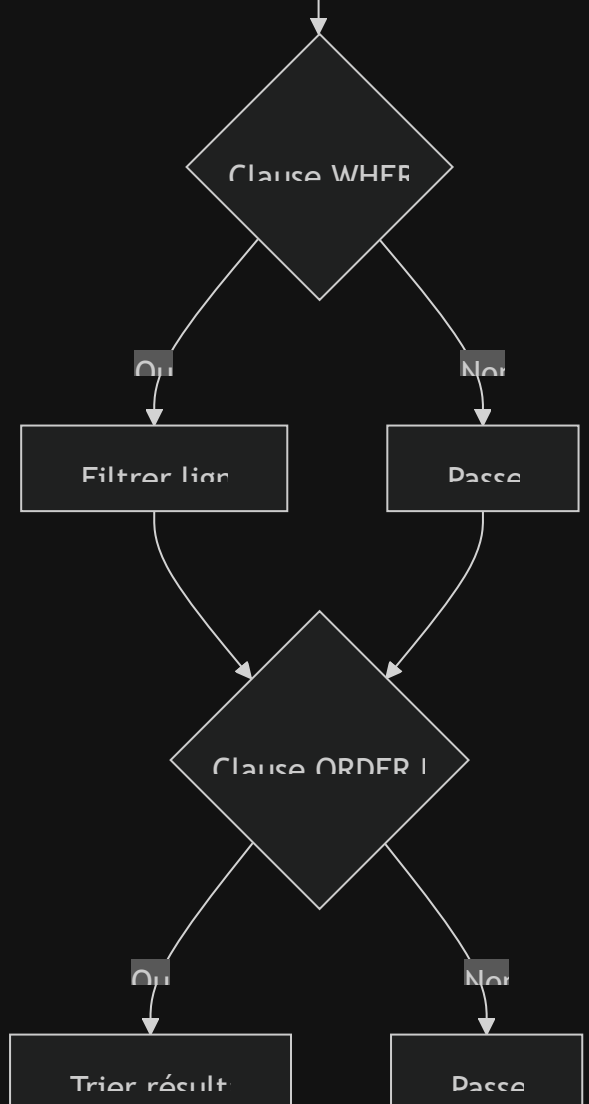
```
SELECT DISTINCT ville FROM Clients;
```

- Renommer colonnes avec alias :

```
SELECT nom AS NomClient, age AS AgeClient FROM Clients;
```

- Calcul simple avec fonction/expression :

```
SELECT nom, age + 1 AS age_lan_prochain FROM Clients;
```


Clause WHER
Ou
Non
Filtrer ligr
Passe
Clause ORDER I
Ou
Non
Trier résult
Passe

Sources utilisées

- Documentation PostgreSQL : SELECT
- W3Schools : SQL SELECT Statement
- TutorialsPoint : SQL Select Statement
- DigitalOcean : Introduction to SQL SELECT Statement

2-Requêtes SQL fondamentales

1-Les commandes de base SQL

2-Ajout de données avec INSERT

La commande INSERT en SQL permet d’insérer de nouvelles données dans une table. Elle est indispensable pour alimenter une base relationnelle avec des enregistrements.

Syntaxe de base de la commande INSERT

```
INSERT INTO table_name [(colonne1, colonne2, ... )]
VALUES (valeur1, valeur2, ... );
```

- `table_name` : nom de la table cible.
- Liste des colonnes optionnelle si l'ordre des valeurs correspond aux colonnes de la table.
- `VALUES` : valeurs à insérer, dans l'ordre des colonnes indiquées.

Exemple simple d’insertion

Table **Employe** :

id (serial)	nom	prenom	age	salaire

Insertion d’un nouvel employé :

```
INSERT INTO Employe (nom, prenom, age, salaire)
VALUES ('Dupont', 'Alice', 30, 3500.75);
```

La colonne `id` auto-incrémentée est remplie automatiquement.

Insertion de plusieurs lignes en une commande

```sql
INSERT INTO Employe (nom, prenom, age, salaire) VALUES
('Martin', 'Bob', 45, 4200.00),
('Leroy', 'Claire', 28, 3200.50);
```

Optimise les performances comparé à plusieurs commandes séparées.

Insérer des données issues d’une autre requête

```sql
INSERT INTO Employe_archives (nom, prenom, age, salaire)
SELECT nom, prenom, age, salaire FROM Employe WHERE age > 40;
```

Permet de transférer ou dupliquer des données selon un critère.

Exemples pratiques

- Insérer avec valeurs par défaut :

```
INSERT INTO Employe (nom, prenom) VALUES ('Durand', 'Eve');
```

- Insérer en utilisant toutes les valeurs par défaut :

```
INSERT INTO Employe DEFAULT VALUES;
```

Processus INSERT (diagramme)

Points importants à retenir

- Respecter l’ordre des colonnes si pas listées explicitement.
- Types des valeurs doivent correspondre aux colonnes.
- Contraintes (clé primaire, not null, unique) sont vérifiées, erreurs levées sinon.
- L’insertion multiple améliore les performances.

Sources utilisées

- Documentation officielle PostgreSQL, INSERT
- W3Schools, SQL INSERT INTO Statement
- TutorialsPoint, SQL Insert Command
- DigitalOcean, How To Use the SQL INSERT Statement

La commande INSERT offre une grande souplesse pour alimenter les tables, en insérant une ou plusieurs lignes, ou en important depuis d’autres requêtes. Elle est l’outil fondamental pour manipuler efficacement les données en SQL.

Requêtes SQL fondamentales

Modification de données avec UPDATE

Commande UPDATE : principe

- Modifie les valeurs existantes dans une table.
- Permet d’ajuster, corriger, mettre à jour les données.
- S’applique selon des conditions précises avec `WHERE`.

Syntaxe de base

```
UPDATE nom_de_la_table
SET colonne1 = valeur1, colonne2 = valeur2, ...
[WHERE condition];
```

- `SET` : colonnes à modifier et leurs nouvelles valeurs.
- `WHERE` (optionnel) : cible les lignes à mettre à jour.
- **Sans `WHERE` : toutes les lignes sont modifiées → risqué !**

Exemples avec la table Employe

id	nom	prenom	salaire
1	Dupont	Alice	3500.75
2	Martin	Bob	4200.00
3	Leroy	Claire	3200.50

Exemple 1 : Mettre à jour le salaire de Bob

```sql
UPDATE Employe
SET salaire = 4500.00
WHERE prenom = 'Bob';
```

Exemple 2 : Augmenter de 5 % le salaire de tous les employés

```sql
UPDATE Employe
SET salaire = salaire * 1.05;
```

Exemple 3 : Modifier plusieurs colonnes pour l’employé id=1

```sql
UPDATE Employe
SET nom = 'Dupond', salaire = 3600.00
WHERE id = 1;
```

Mise à jour conditionnelle avancée

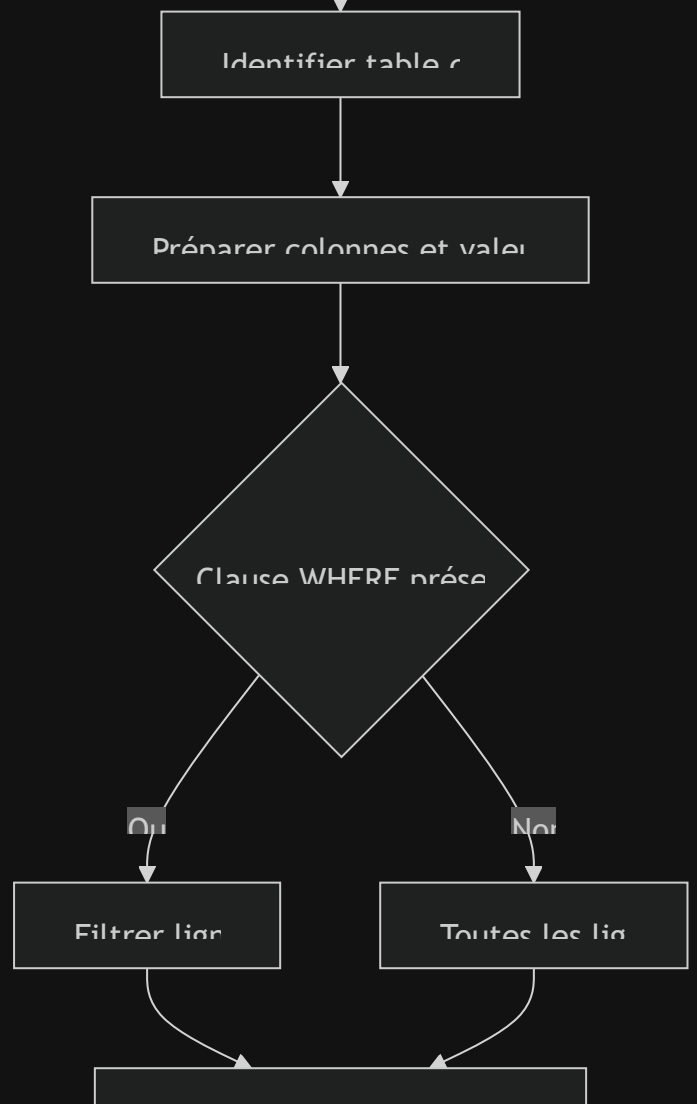
Mettre à jour uniquement les employés dont le salaire est inférieur à 3500 :

```
UPDATE Employe
SET salaire = 3500.00
WHERE salaire < 3500;
```

Utilisation de RETURNING (PostgreSQL)

Récupérer les lignes modifiées en sortie :

```
UPDATE Employe
SET salaire = salaire * 1.10
WHERE id = 3
RETURNING id, nom, salaire;
```


Identifier table c
Préparer colonnes et valeu
Clause WHERE prése
Ou
Non
Filtrer lign
Toutes les lig

Points clés à retenir

- Toujours préférer une clause `WHERE` pour éviter de modifier toutes les lignes par erreur.
- Les valeurs `SET` peuvent inclure calculs, fonctions ou sous-requêtes.
- `RETURNING` est spécifique à PostgreSQL, utile pour récupérer les données mises à jour immédiatement.

Sources

- Documentation officielle PostgreSQL, UPDATE
- W3Schools, SQL UPDATE Statement
- TutorialsPoint, SQL Update Query
- DigitalOcean, How to Use SQL UPDATE Statement

L’instruction UPDATE est un outil puissant pour modifier efficacement vos données. Une bonne maîtrise garantit des mises à jour sûres, ciblées et fiables.

La commande DELETE

- Supprime des enregistrements d’une table en fonction de critères.
- Maîtrise essentielle pour éviter des suppressions massives accidentelles.
- Garantit la cohérence et l’intégrité des données.

Syntaxe générale

```
DELETE FROM nom_de_la_table
[WHERE condition];
```

- `nom_de_la_table` : table ciblée.
- `WHERE` : filtre optionnel pour choisir les lignes à supprimer.
- Sans `WHERE` → suppression de toutes les lignes.

Exemples avec la table Employe

id	nom	prenom	salaire
1	Dupont	Alice	3500.75
2	Martin	Bob	4200.00
3	Leroy	Claire	3200.50

Supprimer l’employé Bob

```sql
DELETE FROM Employe WHERE prenom = 'Bob';
```

Supprimer tous les employés dont le salaire < 3300

```sql
DELETE FROM Employe WHERE salaire < 3300;
```

Supprimer tous les enregistrements (vider la table)

```
DELETE FROM Employe;
```

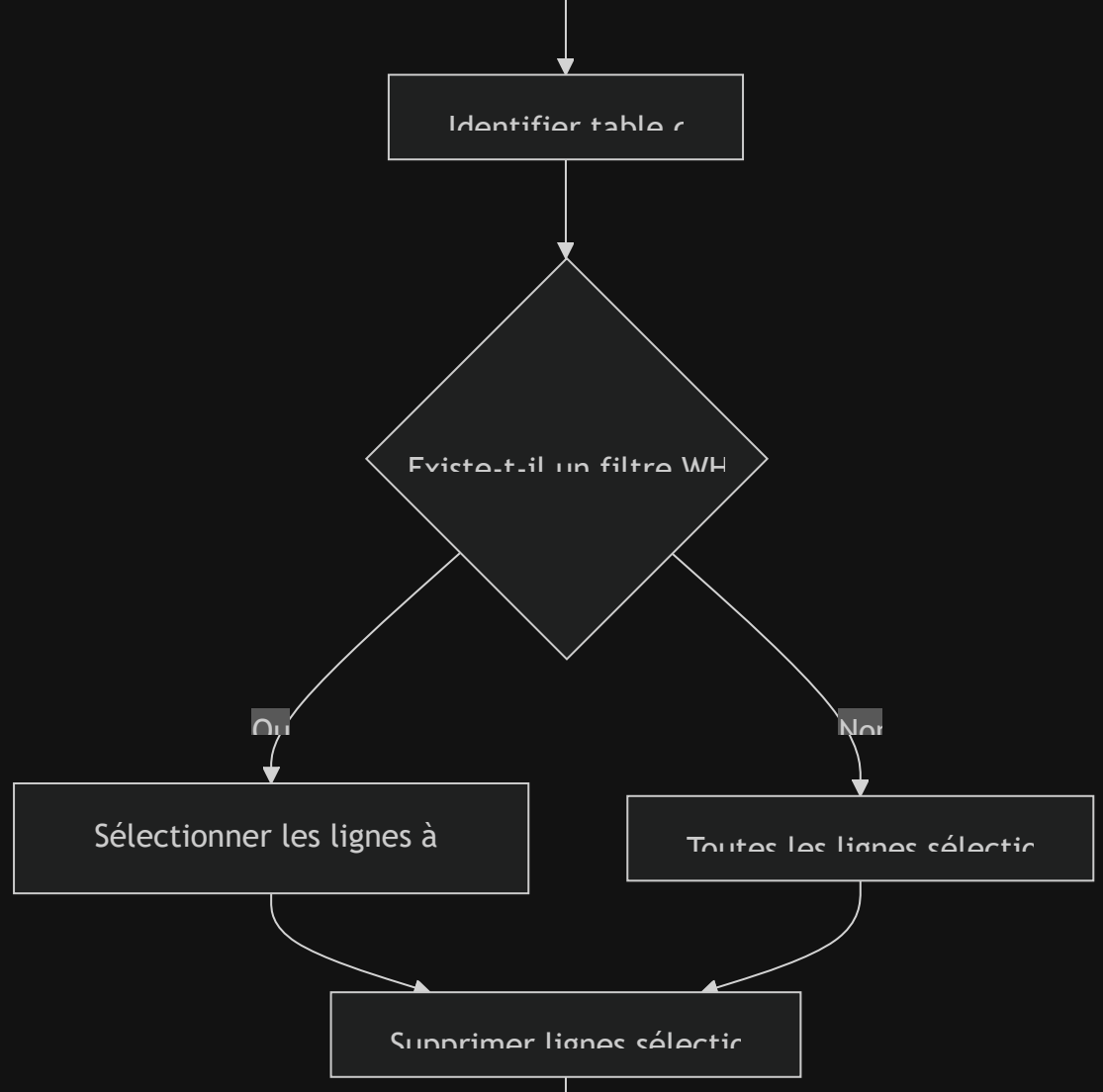
Points importants

- Toujours utiliser `WHERE` pour limiter la suppression.
- Pour vider une table rapidement, `TRUNCATE` est plus performant mais sans filtrage possible.

Suppression avec RETURNING (PostgreSQL)

- Récupère les lignes supprimées pour audit ou traitement.

```
DELETE FROM Employe WHERE salaire < 3300 RETURNING id, nom, salaire;
```


Identifier table c
Existe-t-il un filtre WH
Ou
Non
Sélectionner les lignes à
Toutes les lignes sélectic
Supprimer lignes sélectic

Gestion des contraintes et intégrité

- Suppressions bloquées ou en cascade selon clés étrangères.
- Configurations possibles : `ON DELETE CASCADE`, `RESTRICT`.
- Vérifier les dépendances avant suppression pour éviter les erreurs.

Sources

- Documentation PostgreSQL DELETE
- W3Schools SQL DELETE Statement
- TutorialsPoint SQL Delete Query
- DigitalOcean SQL DELETE Tutorial

Conclusion

La commande **DELETE** assure un contrôle précis sur la suppression des données.

Avec les clauses conditionnelles et fonctionnalités comme `RETURNING`, elle garantit une gestion rigoureuse et sécurisée.

Syntaxe générale

```
SELECT colonnes
FROM table
WHERE condition;
```

- `condition` : simple ou combinée avec opérateurs logiques.

Types d’opérateurs dans WHERE

Opérateurs de comparaison

Opérateur	Description	Exemple
=	égal	age = 30
<> ou !=	différent	ville <> 'Paris'
>	supérieur à	salaire > 2500
<	inférieur à	age < 40
>=	supérieur ou égal à	age >= 18
<=	inférieur ou égal à	salaire <= 3000

Opérateur BETWEEN (inclusif)

```
WHERE age BETWEEN 20 AND 30
```

équivaut à `age >= 20 AND age <= 30`

Opérateur IN

Teste la présence dans une liste :

```
WHERE ville IN ('Paris', 'Lyon', 'Marseille')
```

Opérateur LIKE avec jokers

- `%` : n’importe quelle séquence de caractères
- `_` : un seul caractère

```
WHERE nom LIKE 'Du%'
```

Trouve les noms commençant par "Du"

Opérateur IS NULL / IS NOT NULL

Vérifie la présence ou l’absence d’une valeur nulle.

Combinaison des conditions avec opérateurs logiques

Opérateur	Fonction
AND	toutes les conditions doivent être vraies
OR	au moins une condition est vraie
NOT	inverse la condition

Exemple :

```
WHERE age > 25 AND ville = 'Paris'
```

Exemples d’utilisation

- Clients âgés de plus de 30 ans

```
SELECT * FROM Clients WHERE age > 30;
```

- Clients vivant à Paris ou Lyon

```
SELECT * FROM Clients WHERE ville IN ('Paris', 'Lyon');
```

- Clients dont le nom commence par "Du"

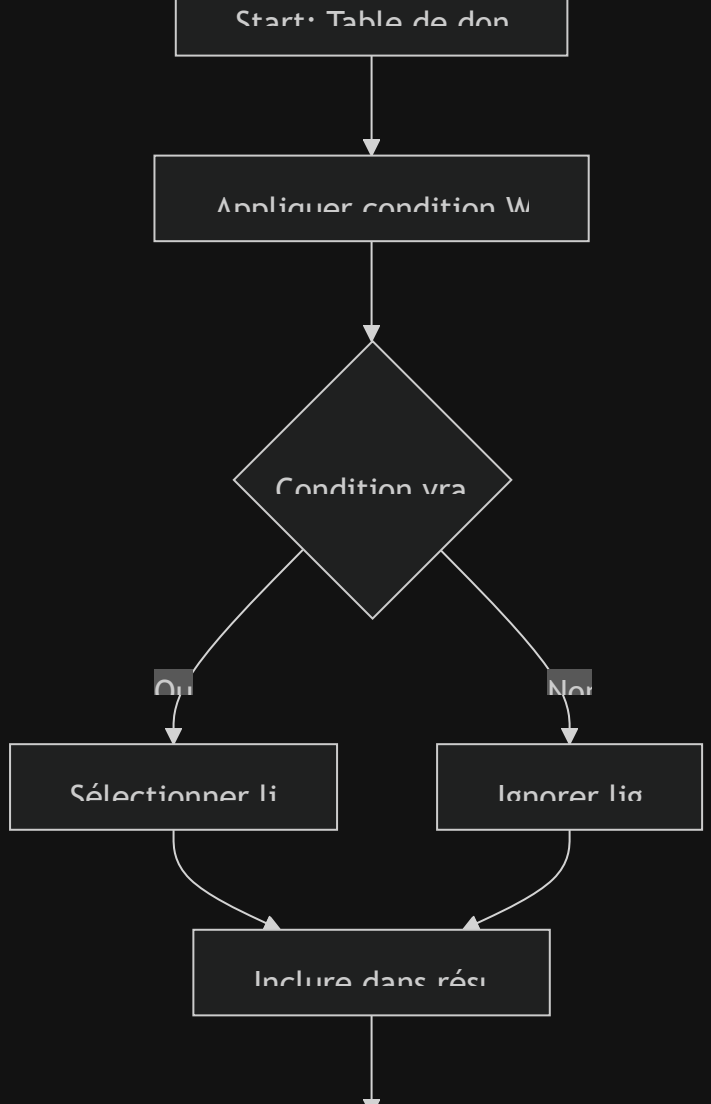
```
SELECT * FROM Clients WHERE nom LIKE 'Du%';
```

- Clients sans numéro de téléphone renseigné

```
SELECT * FROM Clients WHERE telephone IS NULL;
```

Erreur fréquente à éviter

- Omettre la clause `WHERE` dans un `UPDATE` ou `DELETE` modifiera/supprimera toutes les lignes.


Start: Table de don
Appliquer condition W
Condition vra
Ou
Non
Sélectionner li
Ignorer lig
Inclure dans rés

Sources utilisées

- Documentation PostgreSQL, WHERE
- W3Schools, SQL WHERE Clause
- TutorialsPoint, SQL WHERE Clause
- DigitalOcean, An Overview of the WHERE Clause in SQL

La clause WHERE est essentielle pour extraire précisément les données selon des règles définies.

Maîtriser ses opérateurs et combinaisons est la clé d’un filtrage efficace.

2-Requêtes SQL fondamentales

2-Conditions et filtres

2-Opérateurs logiques (AND, OR, NOT)

Opérateurs logiques en SQL

Les opérateurs **AND**, **OR** et **NOT** permettent de combiner ou d’inverser plusieurs conditions dans une clause **WHERE**.

Ils améliorent la précision des filtres pour extraire des données selon des critères multiples ou complexes.

1. Définition et fonction des opérateurs

Opérateur	Description
AND	Vrai seulement si toutes les conditions sont vraies
OR	Vrai si au moins une condition est vraie
NOT	Inverse la valeur logique d’une condition

2. Utilisation et exemples

AND : Toutes les conditions doivent être vraies

```sql
SELECT * FROM Employe
WHERE age > 30 AND ville = 'Paris';
```

Sélectionne les employés ayant plus de 30 ans **ET** vivant à Paris.

OR : Au moins une condition vraie

```sql
SELECT * FROM Employe
WHERE age < 25 OR salaire > 4000;
```

Sélectionne les employés de moins de 25 ans **OU** avec un salaire supérieur à 4000.

NOT : Inverse une condition

```sql
SELECT * FROM Employe
WHERE NOT ville = 'Lyon';
```

Sélectionne les employés dont la ville **n'est pas** Lyon.

3. Combinaisons complexes avec parenthèses

Les parenthèses définissent l'ordre d'évaluation dans des conditions multiples.

```
SELECT * FROM Employe
WHERE (ville = 'Paris' OR ville = 'Lyon') AND salaire > 3000;
```

Sélectionne les employés à Paris **ou** Lyon **avec** un salaire supérieur à 3000.

4. Priorités d’évaluation

- `NOT` a la priorité la plus haute
- `AND` est évalué avant `OR`
- Utiliser les parenthèses pour clarifier l’ordre et éviter les erreurs

5. Exemple complet

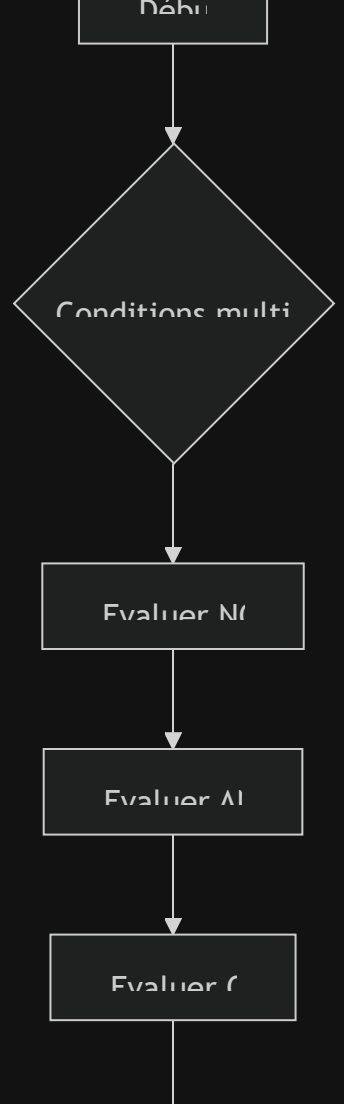
Table **Employe** :

id	nom	ville	age	salaire
1	Dupont	Paris	35	3500.75
2	Martin	Lyon	22	4200.00
3	Leroy	Marseille	45	3200.50

Requête :

```
SELECT * FROM Employe
WHERE (ville = 'Paris' OR ville = 'Lyon') AND NOT age < 30;
```

Résultat : Dupont (Paris, 35 ans) est sélectionné, Martin (Lyon, 22 ans) est exclu.


Conditions multi
Evaluer N(
Evaluer Al
Evaluer (

7. Sources utilisées

- Documentation officielle PostgreSQL, Logical Operators
- W3Schools, SQL AND, OR, NOT
- TutorialsPoint, SQL Logical Operators
- DigitalOcean, SQL Logical Operators

La maîtrise des opérateurs logiques **AND**, **OR** et **NOT** permet de créer des conditions SQL précises et flexibles, facilitant la sélection des données ciblées dans des requêtes complexes.

2-Requêtes SQL fondamentales

2-Conditions et filtres

3-Tri avec ORDER BY

La clause **ORDER BY** en SQL permet de trier les résultats d'une requête selon une ou plusieurs colonnes, dans l'ordre croissant ou décroissant. Elle est utilisée pour organiser la présentation des données et faciliter leur analyse.

Syntaxe de base

```sql
SELECT colonnes
FROM table
[WHERE condition]
ORDER BY colonne1 [ASC | DESC], colonne2 [ASC | DESC], ... ;
```

- `ORDER BY` définit la ou les colonnes sur lesquelles s'effectue le tri.
- `ASC` (ascending) : tri croissant (par défaut).
- `DESC` (descending) : tri décroissant.

Exemples concrets

Trier les employés par nom en ordre alphabétique

```
SELECT * FROM Employe
ORDER BY nom ASC;
```

Trier par salaire décroissant

```
SELECT * FROM Employe
ORDER BY salaire DESC;
```

Tri multiple : par ville croissante, puis âge décroissant

```
SELECT * FROM Employe
ORDER BY ville ASC, age DESC;
```

Tri avec colonnes calculées ou expressions

Tri possible sur expressions ou alias.

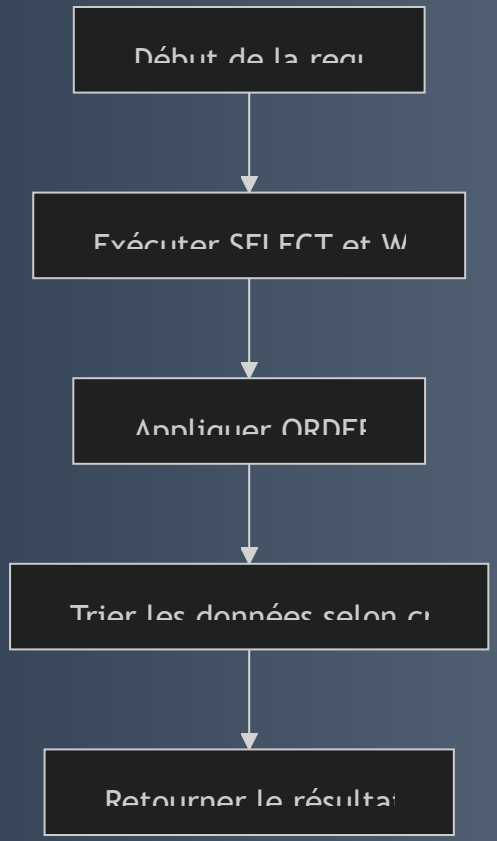
```
SELECT nom, prenom, salaire * 12 AS salaire_annuel
FROM Employe
ORDER BY salaire_annuel DESC;
```

Utilisation de la position des colonnes dans ORDER BY

```sql
SELECT nom, prenom, age
FROM Employe
ORDER BY 3 DESC;
```

Ici, tri selon la 3e colonne affichée (`age`).

Processus ORDER BY



Points importants

- Le tri intervient après la sélection et le filtrage (`SELECT` et `WHERE`).
- Tri croissant par défaut.
- Tri successif si plusieurs colonnes sont spécifiées.
- Le tri peut impacter les performances sur de grandes tables sans index adéquat.

Sources utilisées

- Documentation PostgreSQL, ORDER BY
- W3Schools, SQL ORDER BY Keyword
- TutorialsPoint, SQL ORDER BY Clause
- DigitalOcean, How To Use ORDER BY in SQL

L’utilisation d’**ORDER BY** rend les résultats SQL lisibles et organisés suivant les besoins métier, facilitant ainsi l’analyse et la présentation des données.

Concepts clés du modèle E-A

Entité

- Objet ou concept du monde réel, indépendant dans le contexte.
- Représentée par un rectangle.
- Exemples : Employé, Produit, Client.

Attribut

- Propriété ou caractéristique d’une entité.
- Représenté par une ellipse reliée à l’entité.
- Exemples : id_employe, nom, prénom, date_naissance (pour Employé).

Identifiant (clé)

- Attribut(s) permettant d’identifier de manière unique une entité.
- Souvent souligné dans les diagrammes.
- Exemple : id_employe.

Relation

- Lien entre plusieurs entités.
- Représentée par un losange connecté aux entités concernées.
- Exemple : Travaille_pour reliant Employé et Département.

Types de relations

Type	Description	Exemple
1 - 1	Entité liée à au plus une autre entité	Une personne possède un passeport
1 - N	Entité liée à plusieurs entités d’une autre table	Un département a plusieurs employés
N - N	Plusieurs entités liées à plusieurs entités	Un étudiant inscrit à plusieurs cours

Exemple simplifié de modèle E-A pour une entreprise

- Entités : Employe, Departement, Projet
- Relations :
  - Employé travaille dans un département (1-N)
  - Employé participe à plusieurs projets, projet a plusieurs employés (N-N)

Diagramme Mermaid

Avantages du modèle E-A

- Visualisation claire des données indépendamment du système de gestion.
- Identification simple des entités majeures, attributs et relations.
- Base solide pour la traduction en schéma relationnel.

Sources

- Lucidchart, Entity Relationship Diagram (ERD) Guide
- Oracle, Basic Concepts of ER Model
- Wikipedia, Entity-relationship model
- TutorialsPoint, ER Model

Le modèle entité-association est la première étape pour modéliser graphiquement une base de données relationnelle, facilitant la compréhension des données et leurs interactions avant la conception physique. Sa maîtrise garantit des bases bien pensées, fiables et évolutives.

Modélisation & relations entre tables

Modèle entité-association (E-A)

Conversion d’un schéma E-A en schéma relationnel

Principes généraux de la conversion

Conversion des entités

- Chaque entité devient une **table**.
- Les **attributs** deviennent les colonnes.
- La **clé primaire** de l'entité est la clé primaire de la table.

Exemple

Entité `Employe` avec `id_employe` (PK), `nom`, `prenom`, `date_naissance`

| id_employe (PK) | nom | prenom | date_naissance |

Conversion des relations

Relation 1-1 (un à un)

- Les clés primaires sont liées par une clé étrangère.
- Fusion possible des tables si la relation est obligatoire des deux côtés.

Relation 1-N (un à plusieurs)

- La clé primaire du côté « 1 » devient une clé étrangère dans la table côté « N ».

Relation N-N (plusieurs à plusieurs)

- Création d’une table relationnelle nouvelle.
- Table contient au minimum les clés primaires des deux entités comme clés étrangères formant une clé primaire composée.

Conversion des attributs

- Attributs simples → colonnes simples.
- Attributs composés → décomposés en attributs simples.
- Attributs multivalués → nouvelle table liée par clé étrangère.

Exemple complet

Modèle E-A simplifié

- Entités :
  - `Employe (id_employe, nom)`
  - `Departement (id_departement, nom)`
- Relation 1-N : Un `Departement` a plusieurs `Employe`.

Conversion en schéma relationnel

Employe

id_employe (PK)

nom

id_departement (FK)

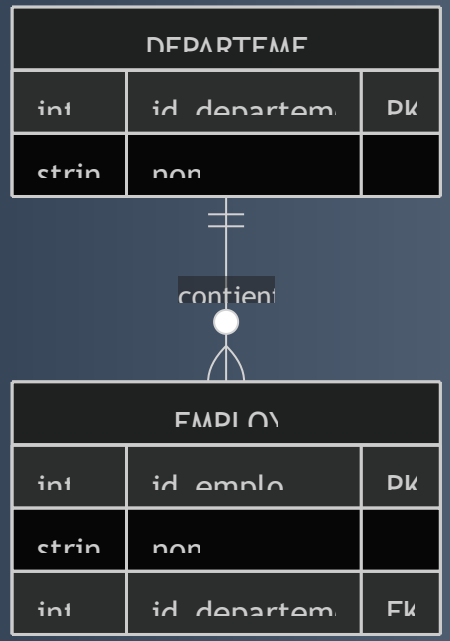
Departement

id_departement (PK)

nom

- `id_departement` dans `Employe` fait référence à `Departement`.

Diagramme Mermaid illustrant la conversion



Remarques supplémentaires

- Pour les relations N-N, la table de relation peut contenir aussi des attributs propres à la relation.
- Une bonne modélisation garantit l’intégrité référentielle via les clés primaires et étrangères.
- Les colonnes doivent respecter les types de données appropriés du modèle E-A.

Sources utilisées

- Oracle, Mapping ER Model to Relational Model
- TutorialsPoint, ER to Relational Mapping
- GeeksforGeeks, Mapping ER Model to Relational Model
- Wikipedia, Entity–relationship model#Mapping to relations

Conclusion

La conversion E-A → relationnel transforme le modèle conceptuel en tables exploitables, organisant données et liens en structures claires pour une gestion efficace dans les bases relationnelles SQL.

Modélisation & relations entre tables

Relations entre tables

Définir les clés primaires et étrangères

Dans une base de données relationnelle, les clés primaires et étrangères sont indispensables pour garantir l’intégrité des données et modéliser les relations entre tables.

Clé primaire (Primary Key)

- Identifie de façon unique chaque enregistrement d'une table.
- Ne peut contenir de valeurs nulles.
- Assure l'unicité des lignes.

Exemple : Table `Employe`

id_employe (PK)	nom	departement
1	Dupont	Marketing
2	Martin	Informatique

- `id_employe` est la clé primaire.

Clé étrangère (Foreign Key)

- Attribut(s) dans une table qui référence(nt) la clé primaire d'une autre table.
- Sert à établir et maintenir les relations entre les tables.
- Assure la cohérence référentielle.

Exemple : Tables `Employe` et `Departement`

Departement
id_departement (PK)
10
20

Table `Employe` :

id_employe (PK)	nom	id_departement (FK)
1	Dupont	10
2	Martin	20

- `id_departement` dans `Employe` est une clé étrangère vers `Departement`.

Règles d’intégrité

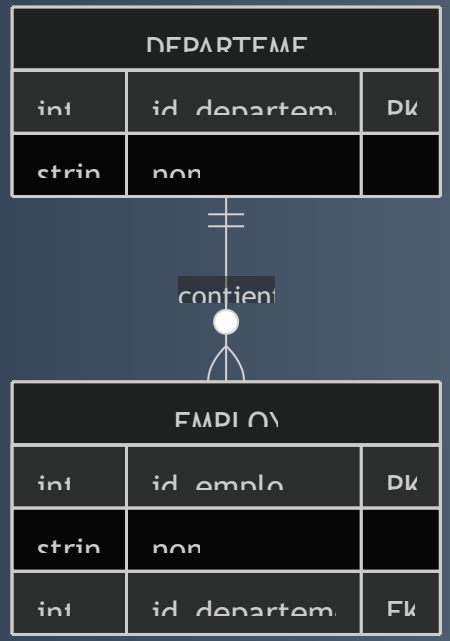
- Une clé étrangère doit correspondre à une valeur existante dans la table référencée.
- Impossible de supprimer un enregistrement référencé sans gérer les dépendances (sauf suppression en cascade).
- La clé primaire est unique et non nulle.

Exemple de déclaration SQL

```sql
CREATE TABLE Departement (
    id_departement INT PRIMARY KEY,
    nom VARCHAR(50)
);

CREATE TABLE Employe (
    id_employe INT PRIMARY KEY,
    nom VARCHAR(50),
    id_departement INT,
    CONSTRAINT fk_departement
        FOREIGN KEY (id_departement)
        REFERENCES Departement(id_departement)
);
```

Diagramme Mermaid illustrant les clés



Sources utilisées

- PostgreSQL Documentation, Primary Keys
- W3Schools, SQL FOREIGN KEY Constraint
- TutorialsPoint, SQL Primary and Foreign Key
- Oracle Docs, Foreign Key Constraints

Les clés primaires garantissent l’unicité des données, les clés étrangères créent des liens cohérents entre tables. Leur bonne définition est essentielle pour l’intégrité de la base et la performance des requêtes.

Modélisation & relations entre tables

Contraintes d’intégrité référentielle

Qu’est-ce que l’intégrité référentielle ?

- Garantie la cohérence et validité des données entre tables liées.
- Utilise les clés étrangères pour éviter les liens invalides.
- Protège la base de données des anomalies.

Définition précise

Toute valeur d’une clé étrangère doit correspondre à une valeur existante dans la clé primaire de la table référencée.

Principaux mécanismes

- **RESTRICT** : Empêche suppression/modification d'une ligne si elle est référencée.
- **CASCADE** : Suppression/mise à jour d'une clé primaire entraîne suppression/mise à jour automatique dans la table liée.
- **SET NULL** : Met la clé étrangère à `NULL` si la ligne référencée est supprimée ou modifiée.
- **SET DEFAULT** : Remplace la clé étrangère par une valeur par défaut spécifiée.

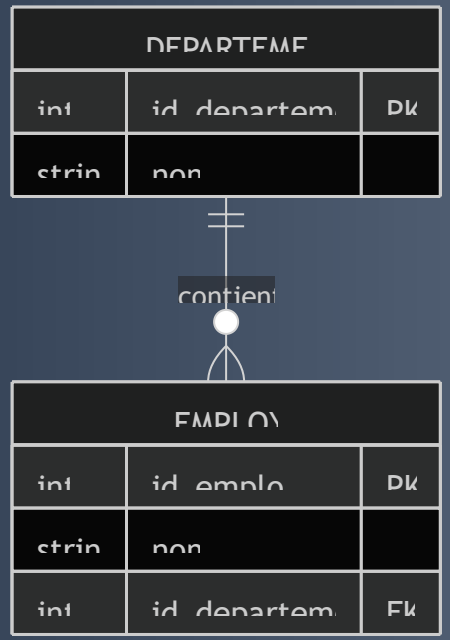
Exemple SQL avec intégrité référentielle

```sql
CREATE TABLE Departement (
    id_departement INT PRIMARY KEY,
    nom VARCHAR(50)
);

CREATE TABLE Employe (
    id_employe INT PRIMARY KEY,
    nom VARCHAR(50),
    id_departement INT,
    CONSTRAINT fk_dept
        FOREIGN KEY (id_departement)
        REFERENCES Departement(id_departement)
        ON DELETE CASCADE
        ON UPDATE RESTRICT
);
```

- `ON DELETE CASCADE` : supprime les employés liés à un département supprimé.
- `ON UPDATE RESTRICT` : interdit la modification d'un département s'il est lié à des employés.

Illustration du comportement CASCADE



La suppression d’un département supprime automatiquement tous ses employés liés.

Impact sur la cohérence des données

- Évite les enregistrements orphelins (sans référence valide).
- Facilite la maintenance des données liées.
- Assure la synchronisation des relations entre tables.

Sources utilisées

- PostgreSQL Documentation, Referential Integrity
- W3Schools, SQL FOREIGN KEY Constraint
- Oracle Docs, Integrity Constraints
- TutorialsPoint, SQL Integrity Constraints

L’intégrité référentielle est essentielle pour garantir la fiabilité et la qualité des bases de données relationnelles, en maintenant la validité des relations entre les données au fil des modifications.

Jointures & requêtes complexes

Types de jointures

Jointure interne (INNER JOIN)

Définition et principe

- `INNER JOIN` combine les lignes de plusieurs tables selon une condition de correspondance.
- Retourne uniquement les enregistrements présents dans toutes les tables jointes.
- Si une ligne n’a pas de correspondance dans l’autre table, elle est exclue du résultat.

Syntaxe

```sql
SELECT colonnes
FROM table1
INNER JOIN table2
ON table1.colonne_commune = table2.colonne_commune;
```

Exemple concret

Tables `Employe` et `Departement` :

id_employe	nom	id_departement
1	Dupont	10
2	Martin	20
3	Durand	NULL

id_departement	nom
10	Marketing
20	Informatique

Requête :

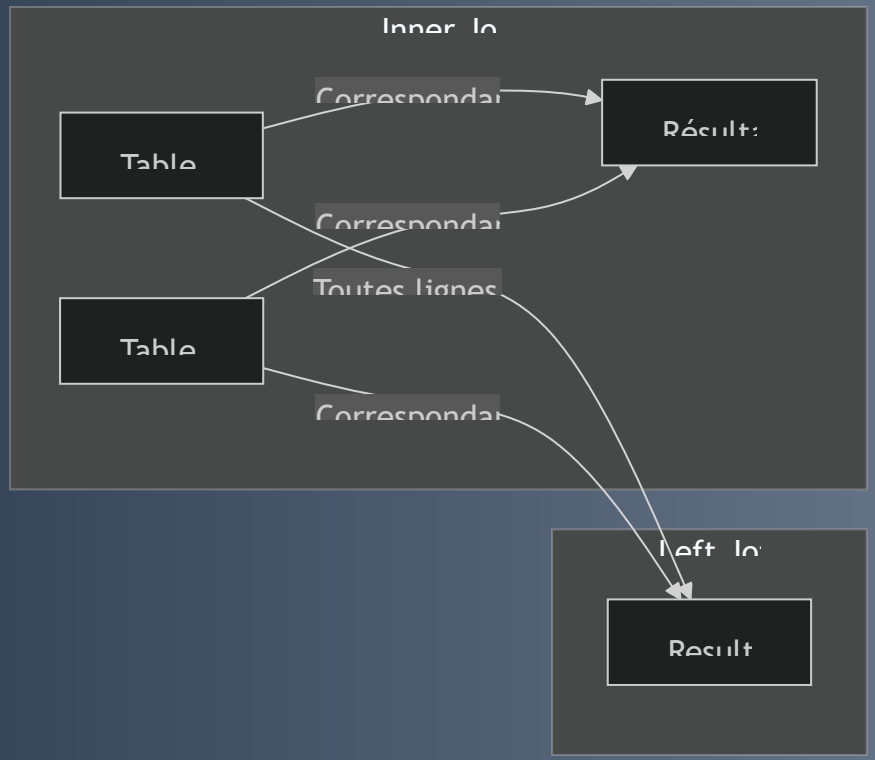
```sql
SELECT e.nom AS employe, d.nom AS departement
FROM Employe e
INNER JOIN Departement d ON e.id_departement = d.id_departement;
```

Résultat :

employe	departement
Dupont	Marketing
Martin	Informatique

- L'employé *Durand* n'apparaît pas, car `id_departement` est `NULL` sans correspondance.

Comparaison avec autres jointures



- `INNER JOIN` sélectionne uniquement les données correspondantes dans les deux tables.
- Contrairement au `LEFT JOIN` qui conserve toutes les lignes de la table de gauche.

Précisions complémentaires

- La condition de jointure porte souvent sur les clés primaires et étrangères.
- `INNER JOIN` est équivalent à une clause `WHERE` combinant les tables, mais la syntaxe explicite améliore la clarté.
- Très utilisé pour croiser et filtrer les données dans les bases relationnelles.

Sources utilisées

- SQL Tutorial, INNER JOIN
- W3Schools, SQL INNER JOIN Keyword
- PostgreSQL Documentation, JOIN clauses
- GeeksforGeeks, SQL INNER JOIN

Jointures externes (LEFT, RIGHT, FULL JOIN)

Les jointures externes permettent de conserver tous les enregistrements d'une ou des deux tables, même sans correspondance. Elles complètent l'`INNER JOIN` en incluant les données non appariées.

Types de jointures externes

LEFT JOIN (Jointure externe gauche)

- Toutes les lignes de la table de gauche sont retournées.
- Les colonnes de la table droite sont `NULL` si pas de correspondance.

RIGHT JOIN (Jointure externe droite)

- Toutes les lignes de la table de droite sont retournées.
- Les colonnes de la table gauche sont `NULL` si pas de correspondance.

FULL JOIN (Jointure externe complète)

- Combine LEFT et RIGHT JOIN.
- Toutes les lignes des deux tables sont retournées.
- `NULL` quand il n’y a pas de correspondance.

Exemple avec les tables Employe et Departement

Employe

id_employe	nom	id_departement
1	Dupont	10
2	Martin	20
3	Durand	NULL

Departement

id_departement	nom
10	Marketing
20	Informatique
30	RH

LEFT JOIN

```sql
SELECT e.nom AS employe, d.nom AS departement
FROM Employe e
LEFT JOIN Departement d ON e.id_departement = d.id_departement;
```

employe	departement
Dupont	Marketing
Martin	Informatique
Durand	NULL

Tous les employés sont affichés, même Durand sans département.

RIGHT JOIN

```sql
SELECT e.nom AS employe, d.nom AS departement
FROM Employe e
RIGHT JOIN Departement d ON e.id_departement = d.id_departement;
```

employe	departement
Dupont	Marketing
Martin	Informatique
NULL	RH

Tous les départements sont affichés, même RH sans employé.

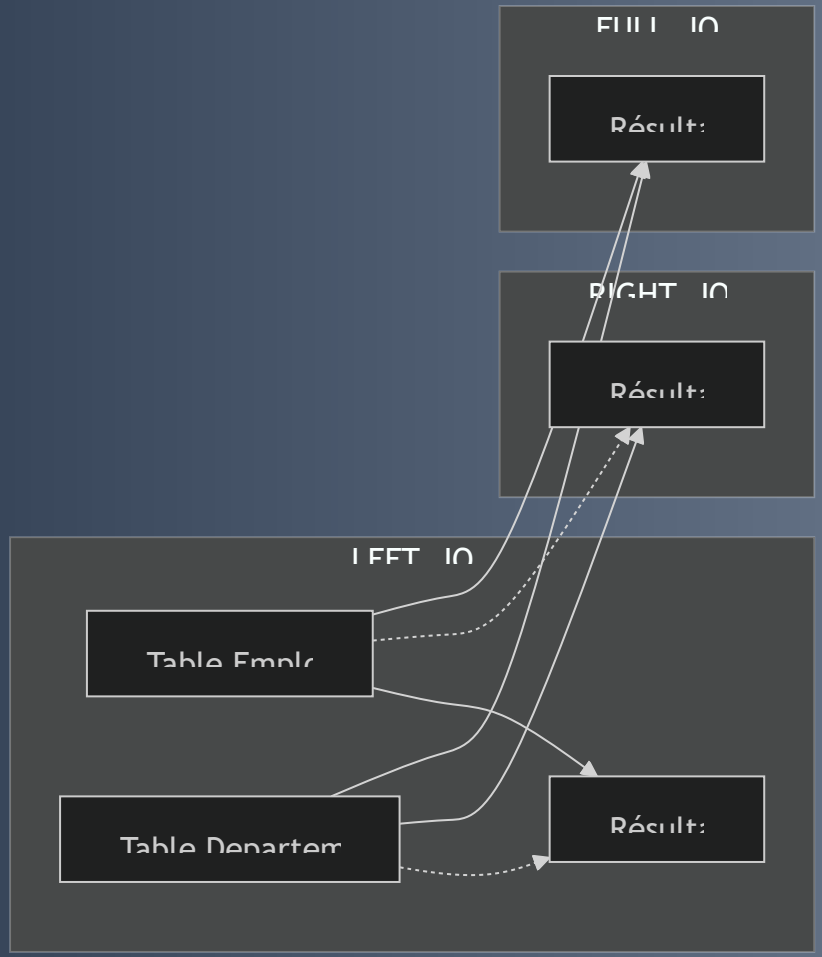
FULL JOIN

```sql
SELECT e.nom AS employe, d.nom AS departement
FROM Employe e
FULL JOIN Departement d ON e.id_departement = d.id_departement;
```

employe	departement
Dupont	Marketing
Martin	Informatique
Durand	NULL
NULL	RH

Affiche tous les employés et tous les départements, avec `NULL` *lorsqu'il n'y a pas de correspondance.*

Visualisation simplifiée



Points importants

- `LEFT JOIN` est le plus utilisé pour récupérer une table principale et ses données associées éventuelles.
- Le choix entre `LEFT`, `RIGHT` ou `FULL` dépend de la priorité aux tables et du besoin métier.
- Ces jointures peuvent introduire des `NULL` dans le résultat.
- Certains SGBD (ex: MySQL < 8.0) ne supportent pas `FULL JOIN`.

Sources utilisées

- W3Schools, SQL LEFT JOIN
- PostgreSQL Documentation, Outer Joins
- GeeksforGeeks, SQL Joins
- SQL Server Docs, Full Outer Join

Les jointures externes élargissent la portée des résultats en conservant des données orphelines d’une table, utiles pour analyser des ensembles incomplets ou comparer des tables partiellement appariées.

Jointures & requêtes complexes

Requêtes imbriquées et agrégations

Sous-requêtes (subqueries)

Qu’est-ce qu’une sous-requête ?

- Requête placée à l’intérieur d’une autre (requête principale).
- Peut apparaître dans `SELECT`, `WHERE`, `FROM`, `HAVING`.
- Son résultat sert à filtrer, calculer ou contrôler des données dans la requête englobante.

Types de sous-requêtes

Type	Description
Sous-requête scalaire	Retourne une seule valeur (un enregistrement, une colonne).
Sous-requête corrélée	Dépend de la requête parente, évaluée pour chaque ligne de la requête externe.
Sous-requête dans `FROM`	Fournit un jeu de résultats temporaire agissant comme une table.

Exemple : Sous-requête simple dans `WHERE`

Trouver les employés du même département que l'employé numéro 1.

```
SELECT nom
FROM Employe
WHERE id_departement = (
    SELECT id_departement
    FROM Employe
    WHERE id_employe = 1
);
```

Exemple : Sous-requête corrélée

Lister les employés ayant un salaire supérieur à la moyenne de leur département.

```
SELECT nom, salaire
FROM Employe e1
WHERE salaire > (
    SELECT AVG(salaire)
    FROM Employe e2
    WHERE e2.id_departement = e1.id_departement
);
```

Exemple : Sous-requête dans `FROM`

Compter les employés par département, puis sélectionner ceux avec plus de 5 employés.

```
SELECT departement, nb_employes
FROM (
    SELECT id_departement AS departement, COUNT(*) AS nb_employes
    FROM Employe
    GROUP BY id_departement
) AS sous_requete
WHERE nb_employes > 5;
```

Concept de sous-requête corrélée (diagramme Mermaid)

Points importants

- Les sous-requêtes corrélées sont coûteuses : évaluées ligne par ligne.
- Les SGBD optimisent mieux les sous-requêtes non corrélées.
- Les jointures (`JOIN`) peuvent offrir de meilleures performances selon les cas.
- Une sous-requête scalaire doit retourner une seule valeur, sinon erreur.

Sources utilisées

- W3Schools, SQL Subqueries
- PostgreSQL Documentation, Subqueries
- Oracle Docs, Subqueries
- SQL Server Docs, Subqueries (Transact-SQL)

Conclusion

Les sous-requêtes permettent d’écrire des requêtes SQL puissantes et flexibles. Maîtriser leur syntaxe offre des possibilités avancées pour exprimer des conditions et organiser clairement ses requêtes complexes.

Jointures & requêtes complexes

Requêtes imbriquées et agrégations

Fonctions d’agrégation (SUM, COUNT, AVG)

Fonctions d’agrégation : définition

- Calculent une valeur unique à partir d’un ensemble de lignes.
- Synthétisent des données numériques ou comptent des éléments.
- Utiles pour résumer et analyser des données dans une table.

Principales fonctions d’agrégation

Fonction	Description	Exemple
`SUM()`	Somme des valeurs numériques	Total des ventes, salaires
`COUNT()`	Nombre d’enregistrements ou de valeurs non nulles	Nombre d’employés, d’articles
`AVG()`	Moyenne des valeurs numériques	Salaire moyen, moyenne des notes

Comportements et particularités

- Ignorent par défaut les valeurs `NULL`.
- Utilisées avec `GROUP BY` pour calculer par groupe.
- `COUNT(*)` compte toutes les lignes, y compris avec `NULL`.
- Renvoient un résultat scalaire : une seule valeur par groupe ou requête.

Exemple : somme des salaires totaux

```
SELECT SUM(salaire) AS total_salaire
FROM Employe;
```

Résultat :

total_salaire
240000

Exemple : nombre d’employés par département

```sql
SELECT id_departement, COUNT(*) AS nb_employes
FROM Employe
GROUP BY id_departement;
```

Résultat :

id_departement	nb_employes
10	5
20	8

Exemple : moyenne des salaires par département

```sql
SELECT id_departement, AVG(salaire) AS salaire_moyen
FROM Employe
GROUP BY id_departement;
```

Résultat :

id_departement	salaire_moyen
10	48000
20	53000

Combinaison de fonctions d’agrégation

```sql
SELECT id_departement,
       COUNT(*) AS nb_employes,
       SUM(salaire) AS masse_salariale,
       AVG(salaire) AS salaire_moyen
FROM Employe
GROUP BY id_departement;
```

Schéma d’agrégation par groupes

```
Error: Parse error on line 4:
...rtement]    SUM[SUM(salaire)]    COUNT
----------------------^
Expecting 'SQE', 'DOUBLECIRCLEEND', 'PE', '-)', 'STADIUMEND', 'SUBROUTINEEND', 'PIPE', 'CYLINDEREND', 'DIAMOND_STOP', 'TAGEND',
'TRAPEND', 'INVTRAPEND', 'UNICODE_TEXT', 'TEXT', 'TAGSTART', got 'PS'
```

Sources

- W3Schools, SQL Aggregate Functions
- PostgreSQL Documentation, Aggregate Functions
- Oracle Docs, Aggregate Functions
- SQL Server Docs, Aggregate Functions (Transact-SQL)

Les fonctions `SUM`, `COUNT` et `AVG` facilitent la synthèse et l'analyse de données numériques en SQL. Leur maîtrise est essentielle pour produire des rapports clairs et pertinents.

Index & performance

Introduction aux index

Concept d’index dans une base relationnelle

Un **index** est une structure de données qui accélère les recherches et tris dans une table, à l’image d’un index dans un livre, pour localiser rapidement des lignes sans parcourir toute la table.

Qu’est-ce qu’un index ?

- Structure souvent basée sur un arbre (ex : arbre B+)
- Stocke les valeurs d’une ou plusieurs colonnes + pointeur vers la ligne correspondante
- Accélère particulièrement les requêtes avec conditions (`WHERE`), jointures, et tris (`ORDER BY`)

Fonctionnement

Au lieu de scanner chaque ligne, le SGBD interroge l’index pour trouver rapidement les emplacements des lignes ciblées.

Exemple : trouver un employé par son `id` est très rapide si un index existe sur la colonne `id`.

Exemple de création d’index en SQL

```sql
CREATE INDEX idx_nom
ON Employe(nom);
```

Cet index optimise des requêtes comme :

```sql
SELECT * FROM Employe WHERE nom = 'Durand';
```

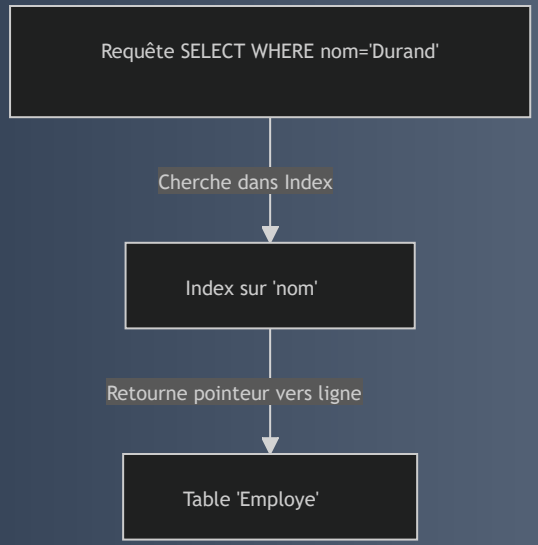
Types courants d’index

Type	Description
Index B-tree	Structure la plus courante, efficace pour recherches d’égalité et d’intervalle
Index hash	Optimisé pour recherche d’égalité uniquement
Index bitmap	Pour colonnes à faible cardinalité, très utilisé en Data Warehouses
Index full-text	Pour recherche sur données textuelles

Impact sur la performance

- **Avantages :** Accélère fortement les sélections, jointures et tris.
- **Inconvénients :** Augmente le coût des insertions, mises à jour et suppressions (mise à jour de l'index).
- Utilisation à adapter selon la fréquence et la nature des requêtes.

Illustration



Sources utilisées

- PostgreSQL Documentation, Indexes
- Oracle Docs, Overview of Indexes
- SQL Server Docs, Indexes
- W3Schools, SQL Indexes

Les index sont essentiels pour améliorer la vitesse des requêtes SQL en évitant le scan complet des tables, mais nécessitent une gestion équilibrée pour ne pas pénaliser les opérations d’écriture.

Index & performance

Introduction aux index

Les index dans une base relationnelle existent sous plusieurs formes adaptées à différents types de requêtes et volumes de données. Les plus courants sont **B-tree** et **hash**, chacun avec ses mécanismes et cas d’usage spécifiques.

Index B-tree (Balanced Tree)

- Structure en arbre équilibré, souvent un **arbre B+**, maintenant les données triées.
- Accès rapide en O(log n) pour recherche, insertion, suppression.
- Gère recherches d'égalité et recherches par intervalle (ex : `WHERE age BETWEEN 20 AND 30`).
- Utilisé par défaut dans la majorité des SGBD (PostgreSQL, MySQL, Oracle, SQL Server).

Exemple de création

```sql
CREATE INDEX idx_nom ON Employe(nom);
```

Utilisation

```sql
SELECT * FROM Employe WHERE nom = 'Durand';
SELECT * FROM Employe WHERE nom BETWEEN 'A' AND 'M';
```

Avantages

- Polyvalent, adapté aux recherches complexes.
- Permet tris rapides (`ORDER BY`).
- Supporte jointures sur colonnes indexées.

Index Hash

- Index basé sur fonction de hachage transformant clé en numéro de "bucket".
- Optimisé pour recherche d'égalité (`=`), très rapide en moyenne.
- Ne supporte pas recherches par intervalle ni tris.
- Supporté par certains SGBD (PostgreSQL), rarement MySQL.

Exemple PostgreSQL

```
CREATE INDEX idx_hash ON Employe USING hash(nom);
```

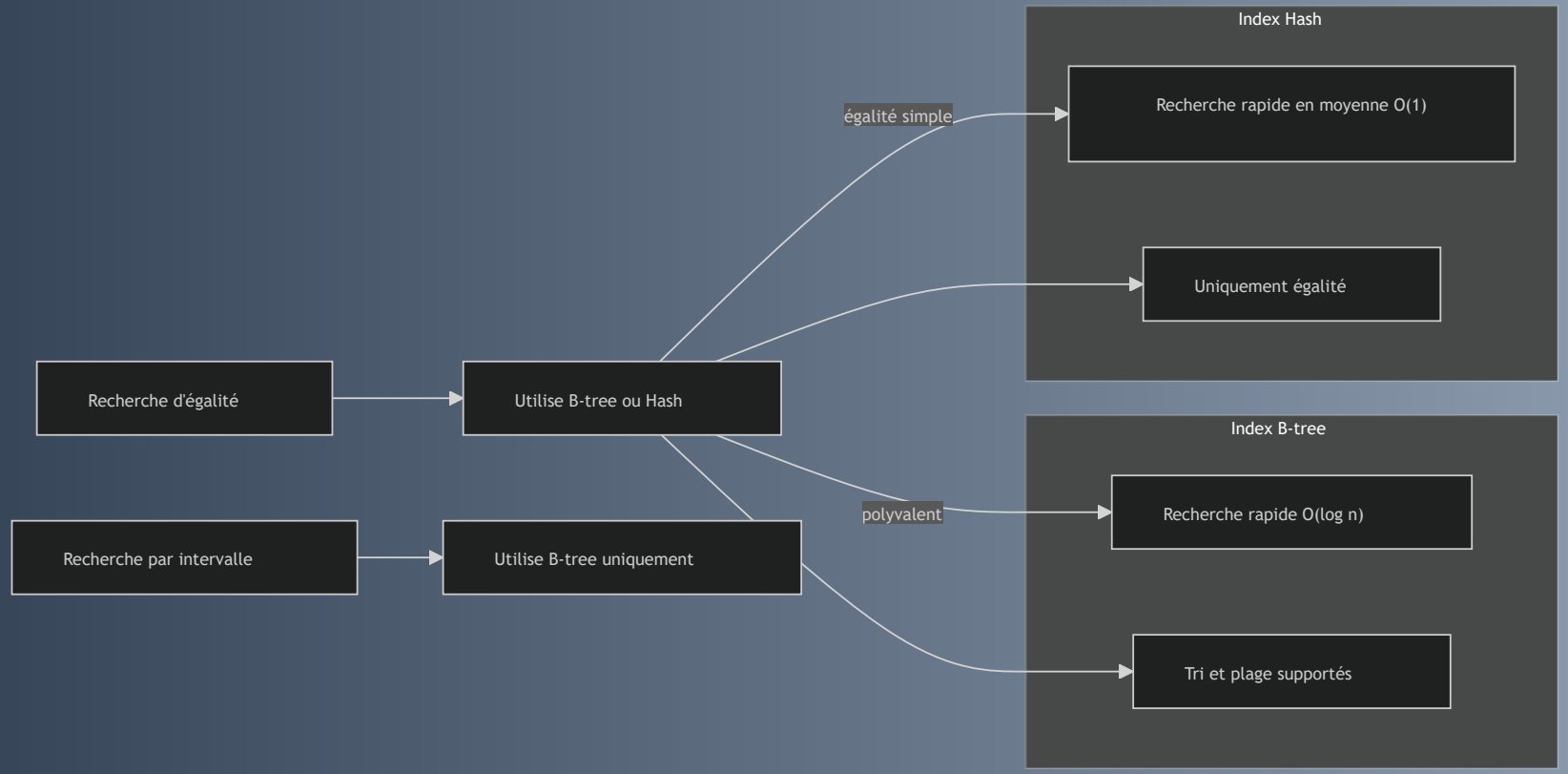
Cas d'utilisation

- Requêtes fréquentes avec égalité stricte (`WHERE id = 42`).
- Inadapté aux intervalles, inégalités, tris.

Comparaison simplifiée

Critère	B-tree	Hash
Recherches supportées	Égalité & intervalles	Égalité uniquement
Performances	Bonnes en général	Très rapides en égalité
Tri & ordre	Oui	Non
Support dans SGBD	Très répandu	Partiel (PostgreSQL)

Vue conceptuelle



Sources utilisées

- PostgreSQL Documentation, Index Types
- Oracle Docs, Index Types
- SQL Server Docs, Index Architecture Guide
- W3Schools, SQL CREATE INDEX

Les index B-tree sont la norme polyvalente capable d’optimiser de nombreuses requêtes, tandis que les index hash offrent des recherches d’égalité très rapides. Choisir entre eux selon les besoins applicatifs optimise les performances.

Analyse des plans d’exécution

L’analyse du **plan d’exécution**

révèle comment un SGBD exécute une requête SQL, montrant le chemin utilisé pour accéder et manipuler les données. Elle met en lumière les points forts et faibles en performance.

Qu’est-ce qu’un plan d’exécution ?

- Description détaillée des étapes internes du SGBD pour une requête.
- Montre les opérations : scans, index seeks, jointures, tris, filtres, agrégations.
- Présenté sous forme textuelle, graphique ou hiérarchique selon l’outil.

Pourquoi analyser un plan d’exécution ?

- Identifier les opérations coûteuses (ex. scans complets, jointures inefficaces).
- Repérer les index utiles ou manquants.
- Vérifier l’optimisation ou nécessité de réécriture.
- Anticiper l’impact lié à l’augmentation des volumes de données.

Outils et commandes selon les SGBD

SGBD	Commande / Outil	Description
PostgreSQL	`EXPLAIN [ANALYZE] <requête>`	Affiche le plan (avec ou sans exécution)
MySQL	`EXPLAIN <requête>`	Plan simplifié, infos sur index utilisés
SQL Server	`SET SHOWPLAN_ALL ON` / SSMS plan graphique	Plan textuel ou graphique
Oracle	`EXPLAIN PLAN FOR <requête>` + `DBMS_XPLAN.DISPLAY`	Visualisation détaillée du plan

Exemple en PostgreSQL

```
EXPLAIN ANALYZE
SELECT * FROM Employe WHERE nom = 'Durand';
```

Sortie simplifiée :

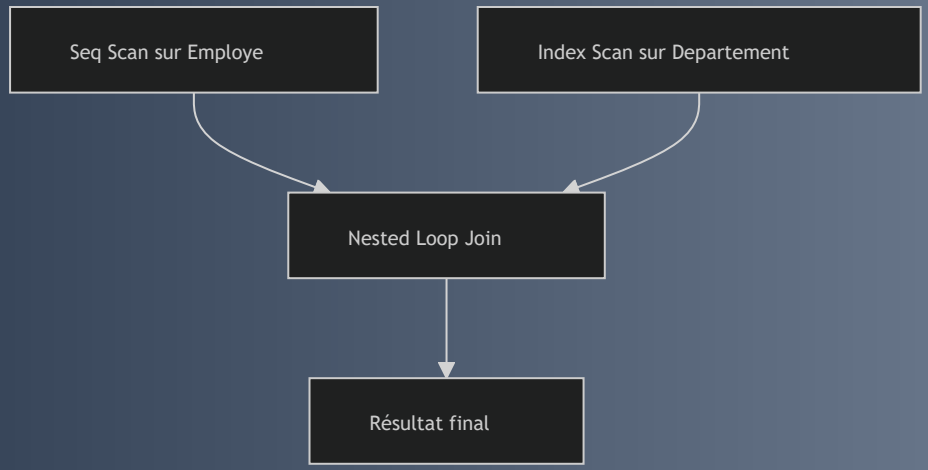
```
Index Scan using idx_nom on Employe  (cost=0.15..8.27 rows=1 width=100)
  Index Cond: (nom = 'Durand')
Planning Time: 0.1 ms
Execution Time: 0.5 ms
```

- *Index Scan* utilise l'index `idx_nom`.
- *Cost* : estimation du coût d'exécution.
- *Rows* : nombre estimé de lignes retournées.
- *Execution Time* : temps réel consommé.

Opérations fréquentes dans les plans

Opération	Description	Impact sur performance
Seq Scan	Parcours complet de la table	Coût élevé sur grandes tables
Index Scan	Recherche via un index	Accès rapide, faible coût
Nested Loop	Jointure imbriquée ligne à ligne	Très coûteux avec grands volumes
Hash Join	Jointure par table hashée	Efficace pour grandes tables
Sort	Tri des résultats	Peut consommer beaucoup de ressources

Exemple simplifié d’un plan



Ce plan montre une jointure où `Employe` est scannée entièrement, `Departement` via un index.

Optimisation basée sur les plans d’exécution

- Remplacer *Seq Scan* coûteux par *Index Scan* en créant les index adaptés.
- Favoriser *Hash Join* ou *Merge Join* plutôt que *Nested Loop* sur grands ensembles.
- Simplifier ou découper les requêtes complexes.
- Maintenir à jour les statistiques des tables pour de bonnes estimations.

Sources utilisées

- PostgreSQL Documentation, Using EXPLAIN
- MySQL Docs, EXPLAIN Syntax
- Microsoft Docs, Showplan Statements
- Oracle Docs, Explain Plan

Optimisation des requêtes : création d’index efficaces

- Créer un index ne garantit pas toujours une meilleure performance.
- Choisir les colonnes à indexer selon les requêtes est essentiel.
- La structure de l’index doit correspondre aux opérations effectuées.

Comprendre la nature des requêtes

- Identifier les colonnes utilisées en `WHERE`, `JOIN`, `ORDER BY`, `GROUP BY`.
- Repérer les types d’opérations (égalité, intervalle, préfixe).
- Évaluer la sélectivité (proportion de valeurs distinctes) des colonnes.

Choix et types d’index

Index simple

Filtrage fréquent sur une seule colonne.

```
CREATE INDEX idx_nom ON Employe(nom);
```

Index composite

Pour filtres multiples; l’ordre des colonnes dans l’index doit refléter celui de la requête.

```
CREATE INDEX idx_dept_salaire ON Employe(departement_id, salaire);
```

Index couvrant (covering index)

Contient toutes les colonnes nécessaires à la requête, évitant la lecture de la table.

Exemple :

```
SELECT nom, salaire
FROM Employe
WHERE departement_id = 10;
```

Index couvrant adapté :

```
CREATE INDEX idx_cover ON Employe(departement_id, nom, salaire);
```

Types d’index selon l’opération

Opération	Index conseillé	Exemple
Égalité (=)	B-tree ou hash (selon SGBD)	`CREATE INDEX ON Employe(nom);`
Intervalle (`BETWEEN`, `<`, `>`)	B-tree	`CREATE INDEX ON Employe(date_embauche);`
Full-text	Index full-text (ex : GIN)	`CREATE INDEX idx_texte ON Article USING GIN(to_tsvector('fr', contenu));`
Requêtes géographiques	Index spatial (ex : R-tree)	`CREATE INDEX ON Lieux USING GIST(geom);`

Impact cardinalité et sélectivité

- **Haute sélectivité** (beaucoup de valeurs distinctes) : index très efficace
- **Basse sélectivité** (peu de valeurs distinctes) : index peu efficace ou nuisible (ex. colonnes booléennes)

Exemple d’optimisation via index

Requête initiale :

```
SELECT * FROM Employe WHERE sexe = 'F' AND departement_id = 4;
```

- Index simple sur `sexe` peu performant (seulement 2 valeurs).
- Préférer un index composite :

```
CREATE INDEX idx_dept_sexe ON Employe(departement_id, sexe);
```

Conseils supplémentaires

- Éviter la multiplication excessive d’index (impact négatif sur insertions/mises à jour).
- Prioriser les colonnes souvent filtrées, triées ou jointes.
- Mettre à jour régulièrement les statistiques des tables.
- Tester avec `EXPLAIN` ou outils de profilage avant/après création.

Sources utilisées

- PostgreSQL Documentation : Indexing Strategies
- Oracle Docs : Choosing an Index
- SQL Server Docs : Designing Indexes
- Use The Index, Luke! : Indexing Strategically

Recherche Full-Text avec PostgreSQL

Module Full-Text Search (FTS) de PostgreSQL

- Recherche textuelle avancée sur de larges volumes
- Plus performant que `LIKE` grâce à un index spécialisé
- Fonctions linguistiques et syntaxe dédiée intégrées

Concepts clés du Full-Text Search

- **tsvector** : représentation vectorielle d'un document (liste de mots triés avec positions)
- **tsquery** : requête de recherche avec opérateurs (`AND`, `OR`, `NOT`)
- **Dictionnaires et configurations linguistiques** : normalisation, suppression des stopwords, stemming
- **Index GIN** : Generalized Inverted Index pour accélérer les recherches full-text

Configuration minimale

Création colonne tsvector

```
ALTER TABLE Article
ADD COLUMN contenu_tsv tsvector;
```

Remplissage de la colonne tsvector

```
UPDATE Article
SET contenu_tsv = to_tsvector('french', contenu);
```

ou colonne générée automatiquement (PostgreSQL 12+)

```
ALTER TABLE Article
ADD COLUMN contenu_tsv tsvector GENERATED ALWAYS AS (to_tsvector('french', contenu)) STORED;
```

Création de l’index GIN

```
CREATE INDEX idx_contenu_tsv ON Article USING GIN(contenu_tsv);
```

- Index spécialisé pour optimiser la recherche full-text sur `contenu_tsv`

Requête de recherche basique

```sql
SELECT id, titre
FROM Article
WHERE contenu_tsv @@ to_tsquery('french', 'base & données');
```

- `@@` : opérateur "correspond à la requête full-text"
- Recherche documents contenant "base" ET "données"

Exemple complet

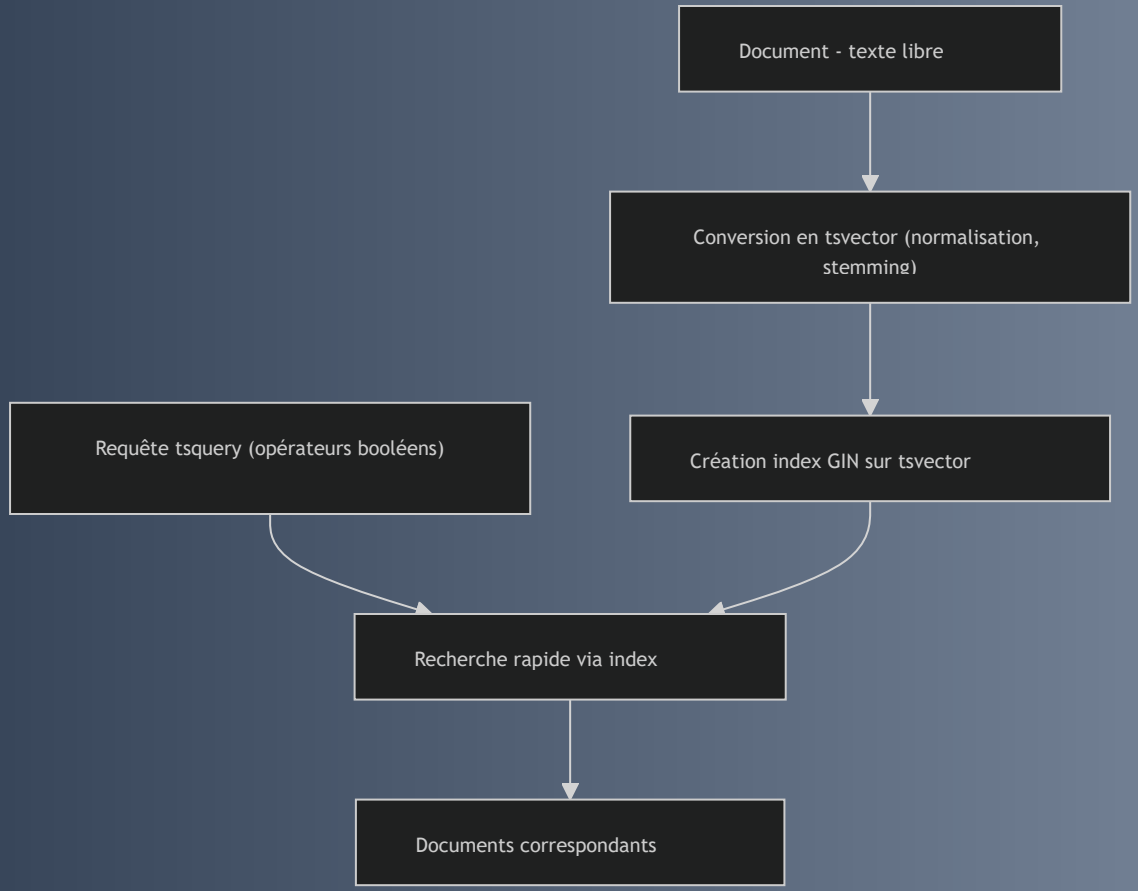
```sql
CREATE TABLE Article (
  id SERIAL PRIMARY KEY,
  titre TEXT,
  contenu TEXT,
  contenu_tsv tsvector GENERATED ALWAYS AS (to_tsvector('french', contenu)) STORED
);

INSERT INTO Article (titre, contenu) VALUES
('Introduction SQL', 'Le langage SQL est utilisé pour manipuler les bases de données.'),
('Base de données avancée', 'Les index améliorent la performance des requêtes.');

CREATE INDEX idx_contenu_tsv ON Article USING GIN(contenu_tsv);

SELECT id, titre
FROM Article
WHERE contenu_tsv @@ to_tsquery('french', 'base & données');
```

Workflow Full-Text Search



Bonnes pratiques

- Choisir la configuration linguistique adaptée (`french`, `english`, etc.)
- Utiliser une colonne `tsvector` matérialisée (générée ou trigger) pour la performance
- Exploiter les opérateurs booléens `&` (AND), `|` (OR), `!` (NOT)
- Mettre à jour les statistiques et analyser le plan d'exécution (`EXPLAIN`) pour vérifier l'utilisation de l'index

Sources principales

- PostgreSQL Documentation - Full Text Search
- PostgreSQL Tutorial - Full Text Search
- Severalnines - PostgreSQL Full Text Search Guide

Recherche Full-Text & alternatives

Recherche Full-Text avec PostgreSQL

Fonctionnalités clés : to_tsvector, to_tsquery, ts_rank

PostgreSQL intègre des fonctions puissantes pour la recherche textuelle avancée, permettant d’analyser, interroger et classer des textes selon leur pertinence.

Fonctions essentielles

to_tsvector

- Transforme un texte brut en un **tsvector** indexable.
- Applique tokenization, supprime les stopwords, et fait du stemming selon la langue.

Exemple :

```
SELECT to_tsvector('french', 'Les bases de données relationnelles sont complexes.');
```

Résultat :

```
'base':2 'donnée':3 'relationnel':4 'complexe':6
```

to_tsquery

- Crée un **tsquery** à partir d'une chaîne textuelle booléenne.
- Opérateurs : `&` (ET), `|` (OU), `!` (NON).

Exemple :

```
SELECT to_tsquery('french', 'base & donnée');
```

Cherche les documents contenant *base* ET *donnée*.

ts_rank

- Calcule un score de pertinence entre un document (`tsvector`) et une requête (`tsquery`).
- Sert à classer les résultats selon leur correspondance.

Exemple simple

```
SELECT contenu,
       ts_rank(
         to_tsvector('french', contenu),
         to_tsquery('french', 'chat & poisson')
       ) AS score
FROM articles
ORDER BY score DESC;
```

Comment le score est calculé ?

`ts_rank` prend en compte plusieurs facteurs :

1. Fréquence des mots

Plus un mot apparaît souvent → plus le score augmente

Exemple :

- "chat mange poisson" → score moyen
- "chat chat chat poisson" → score plus élevé

2. Rareté des mots

Un mot rare a **plus de poids** qu'un mot fréquent

3. Position dans le texte

Les mots apparaissant :

- au début
- ou proches les uns des autres ➡️ augmentent la pertinence

4. Poids des champs (très important)

Il est possible de donner plus d'importance à certaines parties :

```
setweight(to_tsvector(titre), 'A') ||
setweight(to_tsvector(contenu), 'B')
```

Puis :

```
ts_rank(vector, query)
```

Les mots dans le **titre** comptent plus que dans le contenu

Variante : `ts_rank_cd`

Il existe une version améliorée :

```
ts_rank_cd( ... )
```

CD = *Cover Density*

Elle favorise :

- les mots **proches les uns des autres**
- les résultats plus “compacts”

Souvent plus pertinent pour un moteur de recherche type Google

Exemple concret

Imaginons :

contenu	score
"chat mange poisson"	0.8
"chat poisson poisson poisson"	0.9
"chien mange croquette"	0.0

Utilisation typique

```
SELECT *,
       ts_rank(to_tsvector(contenu), query) AS rank
FROM articles,
     to_tsquery('chat & poisson') query
WHERE to_tsvector(contenu) @@ query
ORDER BY rank DESC;
```

Exemple d’utilisation combinée

Table `Article(titre TEXT, contenu TEXT)` :

```
SELECT titre,
       ts_rank(to_tsvector('french', contenu), to_tsquery('french', 'base & donnée')) AS rank
FROM Article
WHERE to_tsvector('french', contenu) @@ to_tsquery('french', 'base & donnée')
ORDER BY rank DESC;
```

- `@@` est le filtre full-text « correspond à la requête ».
- Résultats ordonnés par pertinence décroissante.

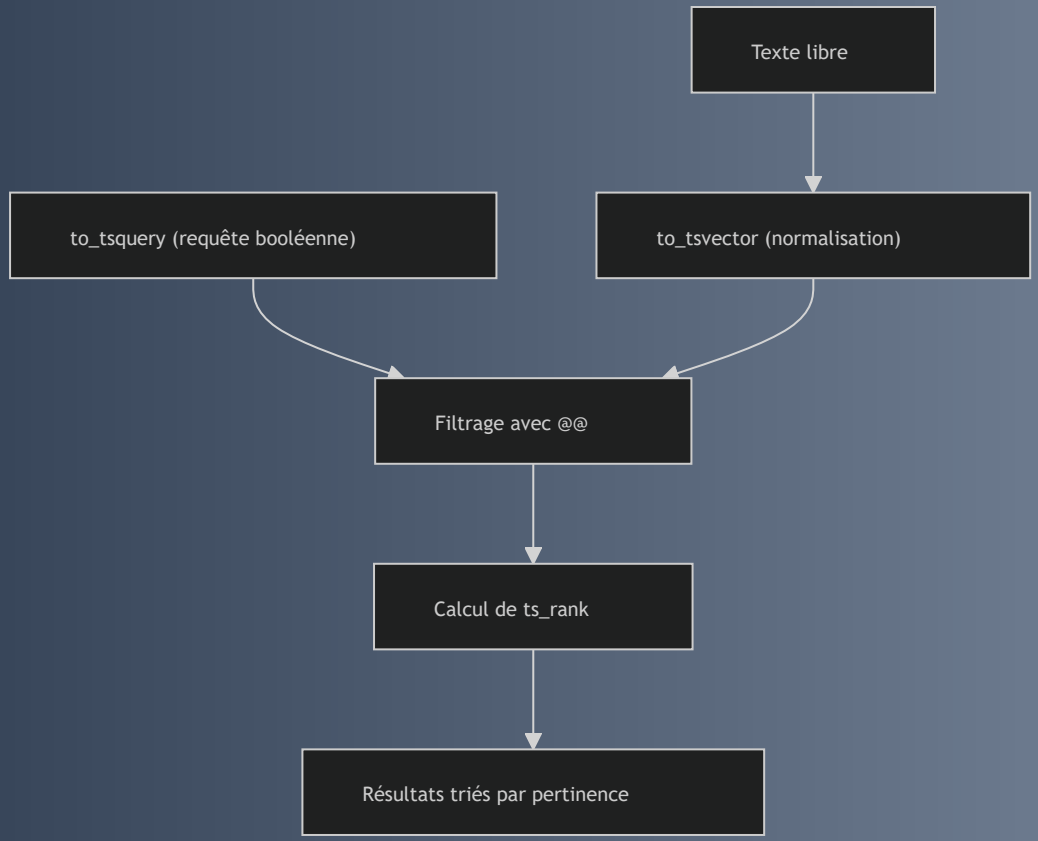
Optimisation avec colonnes matérialisées

Pour éviter recalculs fréquents de `to_tsvector` :

```
ALTER TABLE Article ADD COLUMN contenu_tsv tsvector GENERATED ALWAYS AS (to_tsvector('french', contenu)) STORED;
CREATE INDEX idx_contenu_tsv ON Article USING GIN(contenu_tsv);

SELECT titre, ts_rank(contenu_tsv, query) AS rank
FROM Article, to_tsquery('french', 'base & donnée') query
WHERE contenu_tsv @@ query
ORDER BY rank DESC;
```

Workflow Full-Text Search



Sources de référence

- PostgreSQL Documentation – Full Text Search Functions and Operators
- PostgreSQL Tutorial – PostgreSQL ts_rank Function
- Severalnines – How to Use PostgreSQL Full Text Search Functions

Recherche Full-Text & alternatives

Alternatives et bonnes pratiques

Utilisation de trigrammes, expressions régulières

Au-delà du Full-Text Search classique, PostgreSQL propose les trigrammes et les expressions régulières pour des recherches précises et flexibles, notamment pour la similarité, la gestion des fautes de frappe, et les motifs complexes.

Recherche par trigrammes (pg_trgm)

Principe

- Découpe les chaînes en séquences de 3 caractères (**trigrammes**).
- Mesure la similarité entre chaînes selon l'intersection des trigrammes.
- Utile pour la recherche approximative et les recherches sur sous-chaînes.

Activation de l'extension

```
CREATE EXTENSION IF NOT EXISTS pg_trgm;
```

Création d'un index trigramme

```
CREATE INDEX idx_nom_trgm ON Employe USING GIN (nom gin_trgm_ops);
```

Recherche de similarité

```
SELECT nom FROM Employe WHERE nom % 'Durand';
```

- L'opérateur `%` retourne les noms proches de `'Durand'` selon la similarité trigrammes.

Recherche LIKE optimisée

```
SELECT nom FROM Employe WHERE nom ILIKE '%uran%';
```

- Avec un index trigramme, ce type de recherche utilise les index pour accélérer les performances, même avec des jokers `%` en début de chaîne.

Recherche avec expressions régulières

Principe

- Recherches avancées basées sur des motifs complexes : casse, quantificateurs, groupes, assertions.
- Plus coûteuse en ressources, réservée aux recherches spécifiques.

Syntaxe de base

```
SELECT nom FROM Employe WHERE nom ~ 'Dur.*';
```

- `~` : recherche sensible à la casse.
- `~*` : insensible à la casse.

Exemple

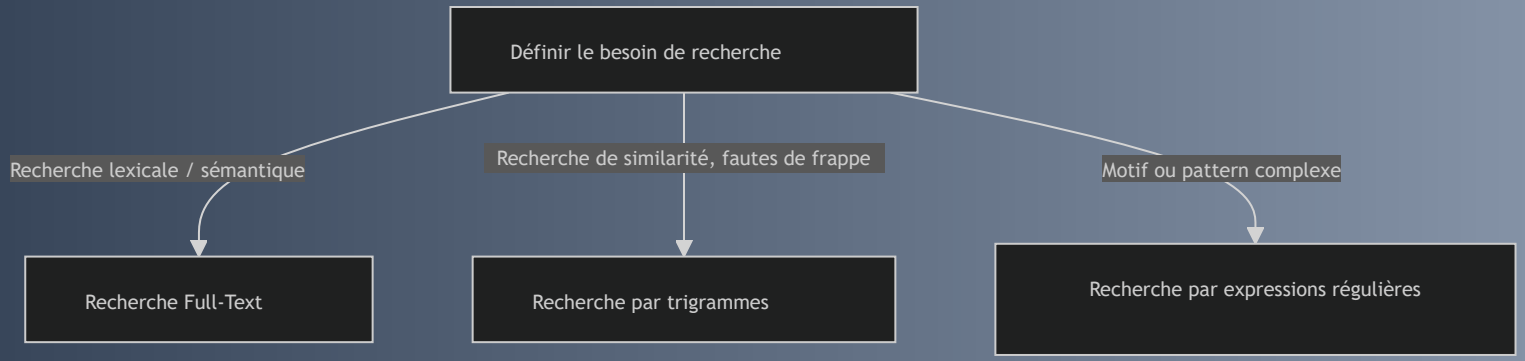
```
SELECT nom FROM Employe WHERE nom ~* '^Dur[a-z]*$';
```

- Recherche les noms commençant par "Dur" suivi de lettres, quelle que soit la casse.

Comparaison rapide des méthodes

Méthode	Usage principal	Performances	Index possible
Full-Text Search	Recherche lexicale avancée, sémantique	Très rapide	GIN sur tsvector
Trigrammes (pg_trgm)	Similarité, sous-chaînes	Rapide pour LIKE et similaires	GIN/GiST avec gin_trgm_ops
Expressions régulières	Motifs complexes	Coûteux à grande échelle	Non

Choix de la méthode selon le besoin



Bonnes pratiques

- Utiliser **Full-Text Search** pour les recherches classiques sur des documents volumineux.
- Activer et exploiter l'extension **pg_trgm** pour optimiser les recherches avec `LIKE '%...%'` et similarités.
- Limiter l'usage des expressions régulières aux cas complexes non couverts par les autres méthodes.
- Toujours créer des index adaptés aux méthodes employées.
- Surveiller utilisation mémoire et performances, car `pg_trgm` peut générer des index volumineux.

Sources utilisées

- PostgreSQL Documentation, pg_trgm module
- PostgreSQL Documentation, Pattern Matching
- Severalnines, Speed Up LIKE Queries with trigram indexes
- Cybertec, Regex vs Full-Text Search

Recherche Full-Text & alternatives

Alternatives et bonnes pratiques

Limites et alternatives au Full-Text Search classique

Limites du Full-Text Search classique dans PostgreSQL

- **Gestion linguistique limitée**

  Stemming et stopwords dépendent des configurations (ex. `french`, `english`), difficultés avec mots composés, synonymes, conjugaisons complexes, langues non couvertes.

- **Absence de recherche approximative (fuzzy search)**

  Pas de gestion des fautes de frappe ou variantes proches, pas de distance de Levenshtein intégrée.

- **Contrôle limité de la pertinence**

  Scores basiques (`ts_rank`), sans pondérations complexes ou apprentissage automatique.

- **Impossibilité de recherche en champ libre pour certains cas**

  Nécessite transformation en `tsvector`, contraignant pour colonnes non optimisées. Recherche sur contenus non textuels non supportée.

Alternatives pour compléter ou remplacer le Full-Text Search

- **Extension pg_trgm : recherche floue**

  Recherche approximative par trigrammes, index GIN/GiST. Utile sur colonnes texte pour erreurs typographiques.

```
CREATE EXTENSION pg_trgm;
CREATE INDEX ON documents USING GIN (contenu gin_trgm_ops);

SELECT * FROM documents WHERE contenu % 'Postgres';
```

- **Expressions régulières et LIKE**

  Flexible mais lent et non indexé (sauf trigrammes activés). Adapté à motifs simples et volumes faibles.

- **Bases noSQL dédiées (ex : MongoDB Atlas Search)**

  Moteurs full-text optimisés intégrés, adaptés aux architectures hybrides.

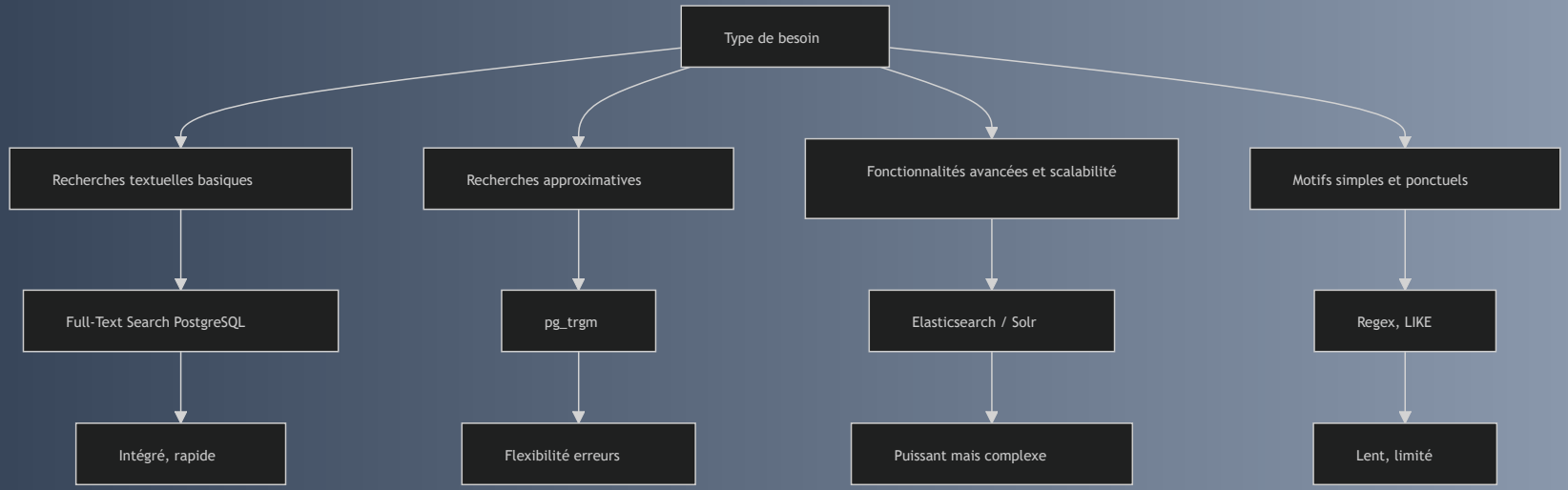
- **Index externes : Elasticsearch, Solr**

  Recherche full-text avancée, index distribués, analyse sémantique. Fonctionnalités avancées : synonymes, facets, typo tolerance, scoring sophistiqué.

Comparaison synthétique des approches

Critère	Full-Text Search PostgreSQL	pg_trgm	Elasticsearch / Solr	Expressions régulières
Volume de données	Moyen à grand	Moyen	Très grand	Petit à moyen
Tolérance aux fautes	Faible	Élevée	Élevée	Faible
Fonctionnalités sémantiques	Basique	Non	Très riche	Non
Facilité d’intégration	Intégré PostgreSQL	Extension PostgreSQL	Solution tierce, plus complexe	Intégré PostgreSQL

Choix selon besoin



Exemple pratique avec pg_trgm

Recherche approximative sur produit dans PostgreSQL :

```sql
CREATE EXTENSION IF NOT EXISTS pg_trgm;

CREATE TABLE produits (
  id SERIAL PRIMARY KEY,
  nom TEXT
);

CREATE INDEX idx_nom_trgm ON produits USING GIN (nom gin_trgm_ops);

INSERT INTO produits (nom) VALUES ('PostgreSQL'), ('Postgres'), ('PostgreSQL Guide');

SELECT * FROM produits WHERE nom % 'Postgresqll';
```

Comparaison avec Elasticsearch

Fonction	PostgreSQL	Elasticsearch
Stemming	Oui	Oui
Stopwords	Oui	Oui
Fautes de frappe	⚠️ (Levenshtein)	Oui natif
Autocomplétion	❌	Oui
Synonymes	limité	Oui
Pertinence	correcte (`ts_rank`)	excellente

Sources utilisées

- PostgreSQL Documentation, Limitations of Full-Text Search
- PostgreSQL Documentation, pg_trgm module
- Elastic, Why Elasticsearch?
- Severalnines, Full Text Search Alternatives